Stanford CS231n《Deep Learning for Computer Vision》学习笔记

> **课程**：Stanford CS231n *Deep Learning for Computer Vision*（2026 春季版，主页标注 "Stanford – Spring 2026"） **课程主页**：cs231n.stanford.edu；**当季日程**：schedule.html **讲义**：本笔记逐讲阅读的是当季公开 PDF，路径形如 `https://cs231n.stanford.edu/slides/2026/lecture_N.pdf`（Lecture 1 拆成 `lecture_1_part_1.pdf` / `lecture_1_part_2.pdf`）；2025 版同类文件在 `/slides/2025/` 下仍然可访问 **官方补充读物（"Useful Notes"）**：cs231n.github.io，含 `optimization-2`（反向传播）、`convolutional-networks`、`neural-networks-2/3`、`understanding-cnn`、`transfer-learning`、`attention`、`transformers`、`generative-models` 等长文 **作业**：Assignments 页 + 起始代码仓库（`github.com/cs231n/cs231n.github.io` 的 `assignments/2026/` 目录，`assignment{1,2,3}.zip`） **视频**：当季录像走 Canvas/Panopto，仅注册学生可见；**2017 版有完整公开视频**，播放列表 `PL3FW7Lu3i5JvHM8ljYj-zLfQRF3EO8sYv`（见 2017 syllabus） **授课教师（当季主页列出）**：Fei-Fei Li、Ehsan Adeli、Justin Johnson、Zane Durante、Tiange Xiang **材料抓取日期**：2026-11-02。所有具体数字（评分比例、讲次日期、top-5 错误率）均来自该日抓取到的公开文本 **免责**：评分比例、讲次顺序、作业题号**每学期都会变**；正式选课或做作业前请以课程当学期主页为准。文中标"推断"处是我的合理外推，不是官方陈述 **定位**：面向已学过 CS229（ML 推导）与 CS336（Transformer/训练系统）的读者，**不重复基础 ML 推导**，专讲视觉特有的归纳偏置、工程细节与"论文里的数字为什么长这样"

1.1 一句话定位
1.2 先修（主页明示）
1.3 评分（当季主页 + Lecture 1 Part 2 明示）
1.4 讲次地图（2026 春季，据 schedule 页）
1.5 与 CS229 / CS336 的分工
1.6 材料可访问性与本笔记的证据等级
2.1 语义鸿沟（semantic gap）
2.2 六大挑战（讲义逐条列出，是后面所有设计的动机）
2.3 四个视觉特有的归纳偏置
2.4 为什么 kNN 在像素上必死
3.1 动机
3.2 模型
3.3 向量化实现（作业里的第一个坑）
3.4 超参与验证集：四种"设超参"的方式
3.5 实验现象与解读
3.6 常见误区
4.1 动机
4.2 多类 SVM 的 hinge loss
4.3 梯度（含不可导点）
4.4 不可导点怎么处理
4.5 W 不唯一：为什么必须正则
4.6 常见误区
5.1 动机
5.2 模型与损失
5.3 梯度：最漂亮的化简
5.4 概率视角：MLE
5.5 数值实例（讲义原例）
5.6 常见误区
6.1 一张表看完
6.2 三个"同一件事"的等价视角
6.3 该用哪个：一个决策清单
6.4 一个常被忽略的工程点：损失尺度
7.1 正则化到底在做什么

7.2 优化：从随机搜索到梯度
7.3 一阶优化器全家桶
7.4 学习率调度
7.5 二阶方法为什么用不了
7.6 常见误区
8.1 两层网络
8.2 激活函数对比
8.3 一个 20 行的完整训练循环
8.4 常见误区
9.1 为什么需要计算图
9.2 链式法则的两种写法
9.3 四种门的模式
9.4 向量化：雅可比视角
9.5 三个必须能手推的反向传播
9.6 数值梯度检验
9.8 常见误区
10.1 动机：全连接层丢掉了什么
10.2 卷积的精确计算
10.3 池化
10.4 感受野（receptive field）计算
10.5 im2col：把卷积变成 GEMM
10.6 一个 3 层卷积网络的完整前向
10.7 常见误区
11.1 演进主线
11.2 AlexNet（2012）：三件事让它成为分水岭
11.3 VGG（2014）：把"深度"变成可复制的配方
11.4 GoogLeNet / Inception（2014）：宽度上的并行
11.5 ResNet（2015）：这一章最重要的内容
11.6 ViT（2021）：把归纳偏置拿掉
11.7 架构选择的工程建议
11.8 常见误区
12.1 初始化：为什么 std 不能随便设
12.2 数据预处理
12.3 Batch Normalization
12.4 Dropout
12.5 数据增强
12.6 一个"正则化配方"的对照表
12.7 常见误区
13.1 六步法（讲义原文顺序）
13.2 随机搜索 > 网格搜索
13.3 曲线诊断表
13.4 一个可直接套用的训练节奏
13.5 常见误区
14.1 动机
14.2 四种情形（讲义的四象限）
14.3 三种落地方式
14.4 为什么迁移学习有效
14.5 模型动物园
14.6 常见误区
15.1 为什么视觉需要序列模型
15.2 Vanilla RNN
15.3 LSTM 与 GRU
15.4 图像描述：CNN + RNN
15.5 与后续的衔接
15.6 常见误区
16.1 从 seq2seq 的瓶颈说起
16.2 自注意力（self-attention）
16.3 Transformer block 的两个版本
16.4 ViT：把图像当词序列
16.5 三种序列建模的取舍（复习用）
16.6 常见误区
17.1 四个任务的输出空间
17.2 语义分割的三代做法
17.3 目标检测：两阶段 vs 单阶段
17.4 Mask R-CNN：把检测扩展到实例分割
17.5 指标：IoU 与 mAP
17.6 全景分割与评价
17.7 常见误区
18.1 五个层次的可视化
18.2 CAM 的推导（讲义给了完整式子）
18.3 对抗样本与 DeepDream
18.4 这些方法能信吗
18.5 常见误区
19.1 问题的规模
19.2 五种融合策略
19.3 3D 卷积的代价
19.4 常见误区
20.1 三种并行

20.2 通信的物理尺度（讲义给的 Llama3 案例）
20.3 显存都花在哪
20.4 大 batch 的两个陷阱
20.5 常见误区
21.1 动机
21.2 对比学习：SimCLR
21.3 自蒸馏：DINO
21.4 掩码重建：MAE
21.5 一张对照表
21.6 常见误区
22.1 分类图（讲义第 14 讲的划分）
22.2 自回归模型
22.3 VAE
22.4 GAN
22.5 扩散模型（DDPM）
22.6 四种生成模型的对照
22.7 常见误区
23.1 五种形状表示
23.2 从多视角到 3D
23.3 常见误区
24.1 从"一个任务一个模型"到"一个模型多个任务"
24.2 CLIP：对比学习把图像和文本对齐
24.3 视觉–语言模型（VLM）的三种接法
24.4 SAM：可提示分割
24.5 常见误区
25.1 与 CS229：把"统计学习"接到"视觉工程"
25.2 与 CS336：Transformer 从语言搬到图像
25.3 与 CS285（RL）：视觉作为状态
25.4 一条建议的学习顺序
26.1 必须能默写（12 项）
26.2 必须能解释"为什么"（10 项）
26.3 必须见过的实验现象（8 项）
28.1 课程官方页面
28.2 讲义 PDF（逐讲下载并 `pdftotext -layout` 阅读）
28.3 官方长文（Useful Notes）
28.4 作业与代码
28.5 讲义中引用的原始论文（仅列课程明确提及者）
28.6 "推断"标注的汇总

第 1 章 课程概览与学习路线

1.1 一句话定位

CS231n 教的不是"如何调用一个 CNN"，而是**如何把一张像素矩阵变成一个可微的评分函数，并让这个函数在有限数据和有限算力下泛化**。整门课可以拆成三层能力：

层	对应讲次	你应当获得的能力
建模层	2–6	把视觉任务写成"评分函数 + 损失 + 正则"的优化问题；判断一个任务该用 kNN、线性模型还是卷积
训练层	3、6、11	手推梯度、写向量化 numpy、诊断 loss 曲线、选初始化与归一化
任务层	9–10、12–17	知道检测/分割/视频/生成/3D 各自的输出空间与评测口径

它和 CS229 最大的差别不在数学，而在**数据的结构**：CS229 的 $\boldsymbol{x} \in \mathbb{R}^d$ 是无序的特征向量；CS231n 的 $\boldsymbol{x} \in \mathbb{R}^{H \times W \times 3}$ 有二维邻接关系、有平移不变性、有巨大的类内方差。所有"视觉特有"的设计——卷积的权值共享、池化的局部聚合、数据增强的裁剪翻转——都是在把这种结构写进模型，也就是**归纳偏置（inductive bias）**。

1.2 先修（主页明示）

先修	主页原文口径	实际会用到的地方
Python	"Proficiency in Python"，作业全部用 numpy	向量化实现 kNN 距离矩阵、im2col 卷积
微积分 + 线性代数	要求"comfortable taking derivatives"、矩阵向量运算	反向传播的雅可比、SVD 视角下的线性分类器
概率统计	要求理解高斯、均值、标准差	softmax 的 MLE 解释、BN 的批统计、VAE 的 ELBO

主页**没有**要求先修深度学习或计算机视觉。但本笔记假设你已有 CS229 的偏差–方差、正则化、凸优化直觉，以及 CS336 的 Transformer 与混合精度训练知识。

1.3 评分（当季主页 + Lecture 1 Part 2 明示）

项目	占比	明细
Assignments	**45%**	A1 12% + A2 18% + A3 15%
In-Class Midterm	**20%**	线下随堂，2026 版排在 5 月 12 日课内
Final Project	**35%**	Proposal 1% + 3 次 Milestone 各 3%（共 9%）+ Final Report 20% + Poster 5%
Participation	额外 **3%**	额外加分，非必修

其他当季政策（原文要点）：4 个免费 late day、每个作业最多用 2 个、用完后每迟一天扣 25%、Final Report 不能用 late day；作业分"代码部分（自动判分）+ 书面部分（助教批改）"；答疑走 Ed，办公时间有"No Code Show Policy"（不给看代码）。

> 注意：上面这套 12/18/15 的拆分是 **2026 春季**的公开数字。历史上 A1/A2/A3 的权重与题目组成都调整过（例如早期 A1 含 SVM 笔记本，2026 版 A1 的五个 notebook 是 kNN / Softmax / 两层网络 / 图像特征 / 全连接网络）。**以课程当学期主页为准。**

1.4 讲次地图（2026 春季，据 schedule 页）

#	日期	主题	本笔记对应章节
1	3/31	Introduction：视觉史 + 课程概览	第 1–2 章
2	4/02	Image Classification with Linear Classifiers（kNN、线性分类器、softmax）	第 3–5 章
3	4/07	Regularization and Optimization（SGD/Momentum/AdaGrad/Adam/调度）	第 6–7 章
4	4/09	Neural Networks and Backpropagation	第 8–9 章
5	4/14	Image Classification with CNNs（卷积、池化、特征图）	第 10 章
6	4/16	Training CNNs and CNN Architectures（BN、初始化、迁移学习、AlexNet/VGG/ResNet）	第 11–14 章
7	4/21	Recurrent Neural Networks（RNN/LSTM/GRU、captioning）	第 15 章
8	4/23	Attention and Transformers（自注意力、ViT）	第 16 章
9	4/28	Object Detection, Segmentation, Visualizing and Understanding	第 17–18 章
10	4/30	Video Understanding	第 19 章
11	5/05	Large Scale Distributed Training	第 20 章
12	5/07	Self-supervised Learning	第 21 章
13	5/14	Generative Models 1（VAE / GAN / 自回归）	第 22 章
14	5/19	Generative Models 2（Diffusion）	第 22 章
15	5/21	3D Vision（形状表示、神经隐式表示）	第 23 章
16	5/26	Vision and Language（CLIP、VLM）	第 24 章
17	5/28	World Modeling（guest lecture）	第 24 章末
18	6/02	Human-Centered AI	第 25 章

另有 6 次周五 Discussion Section：Python/NumPy、Backprop、Project、PyTorch、RNN & Transformer、Midterm Review。Section 的定位是"比正课更多实操细节"，backprop 与 PyTorch 两次对写作业帮助最大。

1.5 与 CS229 / CS336 的分工

主题	CS229 已讲（不重复）	CS231n 新增的东西
线性分类	逻辑回归、softmax 的 MLE、梯度推导	**多类** hinge loss、W 的视觉/几何解释、W 不唯一性与正则
正则化	L1/L2、偏差–方差	dropout / BN / 数据增强 / 随机深度都算"正则化"
优化	牛顿法、GD 收敛	动量族、逐参数自适应、warmup + 余弦、10^3 倍 batch 的线性缩放规则
模型	广义线性模型、核方法	卷积的**参数共享 + 局部连接 + 平移等变**三件套
生成模型	GMM、EM	VAE 的 ELBO 工程化、GAN 的对抗训练、DDPM 的噪声预测参数化
序列建模	HMM	RNN/LSTM 的 BPTT、captioning 的 test-time sampling

主题	CS336 已讲（不重复）	CS231n 的差异
Transformer	从零实现 attention/FFN/RMSNorm	用在**图像 patch** 上：位置编码的作用变了、patch 是 2D 的、归纳偏置被刻意削弱
训练系统	显存核算、FSDP、activation checkpointing	视觉侧的对应物：大 batch 训练、数据并行 vs 模型并行、混合精度
数据	语料清洗、tokenizer	图像增强是"在线生成新样本"，比文本增强激进得多

1.6 材料可访问性与本笔记的证据等级

材料	可访问性	说明
主页 / schedule / assignments 页	✅ 公开	评分、日期、作业构成均取自此处
当季讲义 PDF（`/slides/2026/`）	✅ 公开	16 讲全部可下载，本笔记逐讲读过
2025 讲义 PDF（`/slides/2025/`）	✅ 公开	与 2026 版高度重合，可作对照
起始代码 zip（`assignments/2026/*.zip`）	✅ 公开	函数签名、测试用例、待实现函数的留空位置均来自此处
官方长文笔记（cs231n.github.io）	✅ 公开	公式的规范写法以此为准
当季录像	🔒 Canvas	仅注册学生
2017 录像	✅ YouTube 公开	内容与当季有差异（无 Transformer/扩散），但反向传播、卷积、检测三讲仍是最好入门

材料	可访问性	说明
作业答案 / 考试题	❌ 不公开	本笔记所有"测试读法"均从公开测试代码反推，不含答案

> 写作约定：凡是讲义/主页上出现的数字，我用正常字体并给出来源；凡是"这门课没讲，但工程上必须知道"的，我标 **（推断）**；凡是历史版本差异，我标 **（版本差异）**。

第 2 章 视觉的归纳偏置：为什么不能把图像摊平喂给 CS229 的模型

2.1 语义鸿沟（semantic gap）

计算机看到的是 $800 \times 600 \times 3$ 的整数张量，取值 $[0, 255]$；人看到的是"一只猫"。这中间没有可硬编码的规则。讲义用一张图说明：早期做法是找边缘（Canny 1986）、找角点，但"猫"这个概念无法用边缘的布尔组合写出来。于是转向**数据驱动（data-driven）**范式：

1. 收集 (x, y) 数据集；
2. 用学习算法训练分类器 f_θ；
3. 在新图像上评估。

这三步看似显然，但它把"写规则"换成了"定义损失 + 优化"，从此所有视觉任务都变成同一个模板：**评分函数 → 损失 → 正则 → 优化**。

2.2 六大挑战（讲义逐条列出，是后面所有设计的动机）

挑战	现象	谁来解决它
视角变化（viewpoint variation）	相机一动，所有像素都变	数据增强（随机裁剪/缩放）、平移等变性、多尺度训练
光照（illumination）	同一物体在强光/背光下像素完全不同	逐通道减均值除方差、颜色抖动、BN
背景杂乱（background clutter）	物体与背景纹理相似	更深的层次特征、注意力、检测里的难负样本挖掘
遮挡（occlusion）	只看到局部	局部特征 + 池化（max pooling 天然对局部缺失鲁棒）
形变（deformation）	姿态变化	数据增强、可变形卷积（课程未讲，推断为进阶方向）

挑战	现象	谁来解决它
类内差异（intraclass variation）	"猫"有无数种长相	大数据 + 容量 + 正则

一句话总结这六条：**同一个语义标签对应像素空间里一个极度分散的流形**。所有视觉归纳偏置都是在给这个流形加先验。

2.3 四个视觉特有的归纳偏置

偏置	数学形式	带来什么	代价
局部性（locality）	每个输出只依赖输入的 $k \times k$ 邻域	参数量从 $HW \cdot H'W'$ 降到 k^2	单层感受野小，需要堆叠
平移等变（translation equivariance）	$f(\text{shift}(x)) = \text{shift}(f(x))$	同一滤波器扫全图，参数共享	丢失绝对位置信息（检测里要靠位置编码/anchor 补回来）
层次性（hierarchy）	边缘→纹理→部件→物体	深层组合出语义	需要足够深度与下采样
多尺度（multi-scale）	图像金字塔 / 特征金字塔	应对物体大小变化	计算量与内存

2.4 为什么 kNN 在像素上必死

讲义给了一个极有说服力的实验：对同一张原图做三种变换——**遮挡（occluded）、整体平移 1 像素（shifted）、加色调（tinted）**，这三张图与原图的**像素 L1/L2 距离完全相同**。也就是说，像素距离把"被挡住一半"和"整体挪一个像素"视为同等程度的"不同"，而这在语义上完全不同。

结论（讲义原话大意）：*kNN with pixel distance is never used*。像素距离不具信息量，原因有三：

1. **维度灾难**：3072 维空间里，任意两点的距离都趋同，最近邻不再"近"；
2. **没有不变性**：距离对平移/光照/遮挡极其敏感；
3. **没有语义**：两个像素向量接近，不代表语义接近。

这条结论是整个课程的分水岭：**要么学一个把像素映射到语义空间的函数（线性/CNN），要么换一种表示（HOG、颜色直方图）再算距离**。A1 的 Q4 就是让你亲手验证后者能把 kNN 从 ~28% 抬到 ~40% 量级（具体数字每次运行不同，以你自己的 notebook 输出为准）。

第 3 章 图像分类 I：最近邻与数据驱动范式的边界

3.1 动机

在引入参数化模型之前，课程先用一个"不学习参数"的算法把数据驱动流程走通：**最近邻（Nearest Neighbor）**。它的价值不在性能，而在于把 train/val/test 三段式、超参选择、交叉验证这三个概念一次性讲清楚。

3.2 模型

训练集 $\{(x_i, y_i)\}_{i=1}^{N}$，$x_i \in \mathbb{R}^D$（CIFAR-10 是 $32 \times 32 \times 3 = 3072$）。预测时找

$$i^\star = \arg\min_i \; d(x, x_i), \qquad \hat{y} = y_{i^\star}$$

kNN 把这一个邻居扩展为 k 个，用多数投票：

$$\hat{y} = \text{mode}\{y_i : i \in \mathcal{N}_k(x)\}$$

两种距离：

$$d_1(a,b) = \sum_j |a_j - b_j|, \qquad d_2(a,b) = \sqrt{\sum_j (a_j - b_j)^2}$$

L1 依赖坐标系（旋转后距离会变），L2 与坐标旋转无关。讲义用 $O=(0,0)$、$A=(1,0)$、$B=(1/\sqrt{2}, 1/\sqrt{2})$ 说明：在 L1 下 $d(O,A) = d(O,B) = 1$ 打平，在 L2 下两者也都是 1，但**决策边界形状不同**：L1 的等距面是菱形（在像素空间里偏好坐标轴方向），L2 是圆。

复杂度：训练 $O(1)$（只是记住），预测 $O(N \cdot D)$。这与我们想要的相反——**训练慢没关系，推理必须快**。工业界的补救是近似最近邻（FAISS 等），讲义也点了这条线。

3.3 向量化实现（作业里的第一个坑）

不要写双重循环。把测试集 $X_{\text{te}} \in \mathbb{R}^{M \times D}$ 与训练集 $X_{\text{tr}} \in \mathbb{R}^{N \times D}$ 的距离矩阵用展开式算：

$$\|a - b\|^2 = \|a\|^2 - 2a^\top b + \|b\|^2$$

```
def compute_distances(X, X_train):
    # X: (M, D), X_train: (N, D) -> (M, N)
    a2 = (X ** 2).sum(axis=1, keepdims=True)          # (M, 1)
    b2 = (X_train ** 2).sum(axis=1, keepdims=True).T  # (1, N)
    d2 = a2 - 2 * X @ X_train.T + b2
    d2 = np.maximum(d2, 0)   # 浮点误差可能让对角线出现 -1e-12
    return np.sqrt(d2)
```

三个必须知道的细节：

1. `np.maximum(d2, 0)` 不是洁癖：10^4 维下 $a^\top a$ 的舍入误差可能让自距离变成 -10^{-9}，`sqrt` 直接 `nan`。
2. 内存：$M = N = 5000$ 时 (M, N) 的 float64 是 200 MB。A1 让你同时写双循环版，就是感受 $O(MN)$ 的内存墙。距离平方单调，只取 `argmin` 时可跳过 `sqrt`。

3.4 超参与验证集：四种"设超参"的方式

方案	做法	为什么错/对
Idea #1	选在**训练集**上最好的	错。$k = 1$ 在训练集上永远 100%
Idea #2	选在**测试集**上最好的	错。等于偷看答案，无法估计泛化
Idea #3	train / val 划分，在 val 上选，最后测一次 test	对。深度学习默认做法
Idea #4	k-fold 交叉验证，各折平均	对，但小数据才用；深度学习中不常用（训练太贵）

讲义给了一张 5-fold 交叉验证选 k 的图，横轴 k、纵轴准确率，误差棒是各折标准差，结论是**这份数据上 $k \approx 7$ 最好**。注意这句话的限定："for this data"——k 没有普适最优值。

3.5 实验现象与解读

- kNN 在 CIFAR-10 上大约 **28%**（$k = 1$，L2），远低于线性分类器的 ~35–40%，更远低于 CNN 的 95%+。
- k 从 1 增大时，决策边界变平滑，准确率先升后降：太小对噪声敏感，太大把不同类混在一起。
- 换 HOG + 颜色直方图特征后，同一个 kNN 能到 ~40%（A1 Q4 的目的）；换 CNN 特征（fc7）能到 ~60%（迁移学习的雏形）。

3.6 常见误区

误区	纠正
"kNN 是非参数方法，所以不需要调参"	k、距离度量、是否归一化都是超参
"像素距离不行，说明 kNN 这个算法不行"	换成有语义的特征后 kNN 立刻可用。**问题在表示，不在算法**
"交叉验证一定比单次划分好"	交叉验证把训练成本乘 k；大数据/深度模型上几乎没人做
"先跑 test 看看效果"	每跑一次 test 就多一次信息泄漏；测试集**只用一次**

第 4 章 图像分类 II：线性 SVM 与 hinge loss

4.1 动机

kNN 的问题：需要存下整个训练集、推理 $O(N)$、没有"学习"出任何可迁移的东西。**参数化方法**把知识压进一组参数 W：

$$f(x, W) = Wx + b, \qquad W \in \mathbb{R}^{C\times D},\ x \in \mathbb{R}^{D},\ b \in \mathbb{R}^{C}$$

CIFAR-10 里 W 是 10×3072，加 b 共 30710 个参数。三个视角（讲义反复强调，考试爱考）：

视角	说法	直觉
代数（algebraic）	$f(x, W) = Wx + b$，每行 W_c 与 x 做内积	每个类一个"模板打分器"
视觉（visual）	把 W 的每一行 reshape 回 $32 \times 32 \times 3$ 当图像看	每类学到一个**模板**（template），像"平均的猫"
几何（geometric）	每个类对应空间中一条超平面，分类 = 看 x 落在哪一侧	线性分类器只能切出**凸的**、**单连通**的区域

4.2 多类 SVM 的 hinge loss

对样本 (x_i, y_i)，分数 $s = f(x_i, W)$：

$$L_i = \sum_{j\neq y_i} \max\left(0,\ s_j - s_{y_i} + \Delta\right)$$

Δ（margin）课程固定取 1。总损失：

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + \lambda \sum_{k}\sum_{l} W_{k,l}^2$$

为什么 $\Delta = 1$ 可以固定？ 因为 W 的尺度可自由缩放：把 $W \to 2W$ 时所有分数翻倍，$\max(0, 2s_j - 2s_{y_i} + 1)$ 与 $\max(0, s_j - s_{y_i} + 1)$ 不等价，但**只要 λ 相应调整，最优解等价**。真正的权衡由 λ 控制，Δ 只是把"间隔"的绝对尺度定死。这是讲义明确点出的"Δ 与 λ 冗余"。

4.3 梯度（含不可导点）

对每个 $j \neq y_i$：

$$\frac{\partial L_i}{\partial s_j} = \begin{cases} 1, & s_j - s_{y_i} + \Delta > 0 \\ 0, & \text{otherwise} \end{cases} \qquad \frac{\partial L_i}{\partial s_{y_i}} = -\sum_{j\neq y_i} \mathbb{1}\left[s_j - s_{y_i} + \Delta > 0\right]$$

于是

$$\nabla_W L_i = \sum_{j \neq y_i} \mathbb{1}[\cdot]\,(x_i \text{ 作为第 } j \text{ 行}) - \left(\sum_{j \neq y_i} \mathbb{1}[\cdot]\right) x_i \text{ 作为第 } y_i \text{ 行}$$

实现时最省事的写法是"**先算计数，再一次性做外积**"：

```
def svm_loss_naive(W, X, y, reg):
    dW = np.zeros_like(W)
    loss = 0.0
    for i in range(X.shape[0]):
        scores = X[i] @ W.T           # (C,)
        correct = scores[y[i]]
        margins = np.maximum(0, scores - correct + 1.0)
        margins[y[i]] = 0.0           # 正确类不计入
        loss += margins.sum()
        # 梯度：每个 margin>0 的类 j 贡献 +x_i 到第 j 行，-x_i 到第 y_i 行
        mask = (margins > 0).astype(float)
        dW += np.outer(mask, X[i])
        dW[y[i]] -= mask.sum() * X[i]
    loss = loss / X.shape[0] + reg * np.sum(W * W)
    dW = dW / X.shape[0] + 2 * reg * W
    return loss, dW
```

注意 `margins[y[i]] = 0` 必须在算 `mask` 之前，否则正确类会被算成"margin 超限"。

4.4 不可导点怎么处理

$\max(0, z)$ 在 $z = 0$ 处不可导。实践上取**次梯度（subgradient）**，即该点任取一侧的导数（通常取 0 或 1）。这对 SGD 无影响：单点取值不影响积分意义上的下降方向，且浮点运算下 z 精确等于 0 的概率为 0。**（推断）** 如果你想严格，可以在实现里用 `>= 0` 还是 `> 0` 都行，但**数值梯度检验会在这点上产生微小差异**，所以要理解 `rel_error` 不是 0 是正常的。

4.5 W 不唯一：为什么必须正则

讲义用一组具体分数演示：若某个 W 让所有 margin 都为 0（$L = 0$），那么 $2W$ 也让所有 margin 为 0（因为 $\max(0, 2z) = 0 \iff \max(0, z) = 0$，当 $z \leq 0$）。所以**满足 $L = 0$ 的 W 有无穷多个，构成一个缩放族**。如何在这些等价解里选一个？

- L2 正则：偏好 $\|W\|$ 小的解，等价于"最大化 margin"（这是 SVM 的经典结论，CS229 讲过对偶）；
- L1 正则：偏好稀疏解，倾向于让 W 的很多元素为 0；
- 两者都让解**唯一**（严格凸时）。

4.6 常见误区

误区	纠正
"hinge loss 有 margin，所以对异常点鲁棒"	恰恰相反：hinge 是**线性增长**的，一个极端错分点会产生巨大 loss；squared hinge 更敏感；softmax 是指数惩罚，更敏感
"Δ 越大分类越严格"	Δ 与 λ 冗余，单独调 Δ 等价于调 λ 的倒数
"线性分类器学到的模板就是类的原型"	模板是"能让该类分数高的方向"，可能由背景/颜色主导（讲义展示了 horse 模板里出现马头朝向的两面）
"线性分类器可以处理 XOR / 同心圆"	不行。讲义明确列了三类 hard case：象限奇偶、环形（$1 \le \|x\|_2 \le 2$）、多模态

第 5 章 图像分类 III：softmax 与交叉熵

5.1 动机

hinge loss 只关心"正确类是否领先 Δ"，不输出概率。很多下游任务（检测的置信度、多标签、蒸馏）需要**校准过的概率**。softmax 把分数变成概率分布。

5.2 模型与损失

$$p_j = \frac{e^{s_j}}{\sum_{k=1}^{C} e^{s_k}}, \qquad L_i = -\log p_{y_i} = -s_{y_i} + \log \sum_k e^{s_k}$$

后者叫 **log-sum-exp** 形式，是数值稳定的写法。整个数据集 $L = \frac{1}{N}\sum_i L_i + \lambda \|W\|^2$。

两个必会的小问题（讲义原题）：

- L_i 的范围：$[0, +\infty)$。最小 0（$p_y = 1$），最大无穷（$p_y \to 0$）。
- 初始化时所有 $s_j \approx 0$，则 $L_i = \log C$。$C = 10$ 时 ≈ 2.302。**这是你调试时的第一个 sanity check**：初始 loss 明显偏离 $\log C$，说明初始化或标签映射有 bug。

5.3 梯度：最漂亮的化简

$$\frac{\partial L_i}{\partial s_j} = p_j - \mathbb{1}[j = y_i]$$

推导（一行）：$\partial(-s_y)/\partial s_j = -\mathbb{1}[j=y]$；$\partial \log \sum_k e^{s_k} / \partial s_j = e^{s_j}/\sum_k e^{s_k} = p_j$。

含义极清晰：**梯度 = 预测概率 − one-hot 标签**。预测对了（$p_y \to 1$）梯度趋 0；预测错了梯度大。这也是为什么 softmax + CE 在框架里是一个 fused kernel——分开算再相乘既慢又容易数值溢出。

```
def softmax_loss_vectorized(W, X, y, reg):
    N = X.shape[0]
    scores = X @ W.T                            # (N, C)
    scores -= scores.max(axis=1, keepdims=True)   # 数值稳定：减行最大值
    exp_s = np.exp(scores)
    probs = exp_s / exp_s.sum(axis=1, keepdims=True)
    loss = -np.log(probs[np.arange(N), y]).mean() + reg * np.sum(W * W)
    dscores = probs.copy()
    dscores[np.arange(N), y] -= 1.0
    dW = dscores.T @ X / N + 2 * reg * W
    return loss, dW
```

减行最大值不改变数学结果（分子分母同乘 e^{-m}），但能避免 e^{1000} 上溢。**这是 A1 里最容易被扣分的细节之一**：不减最大值时，测试用的极端分数会 `inf/nan`。

5.4 概率视角：MLE

$$\prod_i p_{y_i} \implies -\log \prod_i p_{y_i} = \sum_i L_i$$

最小化交叉熵 = 最大化似然。讲义明确写 "See CS 229 for details"，所以这里不重复 MLE 与 KL 的等价性。补充一点 CS229 没强调的：softmax 的参数化是**过参数化**的（给所有 s_j 加同一个常数不改变 p），所以 W 也有平移不变性，这解释了为什么 softmax 的 W 同样不唯一，正则同样重要。

5.5 数值实例（讲义原例）

分数 $[3.2,\ 5.1,\ -1.7]$：$e^{3.2} = 24.5$、$e^{5.1} = 164.0$、$e^{-1.7} = 0.18$，和 $= 188.7$，概率 $[0.13, 0.87, 0.00]$。若真值是 cat，$L_i = -\log 0.13 = 2.04$。注意 frog 的概率显示为 0.00 但**不是 0**（是 9.5×10^{-4}），所以 loss 不会真的无穷。

5.6 常见误区

误区	纠正
"softmax 输出概率，所以可以当置信度直接用"	现代网络普遍**过自信**（over-confident），需要 temperature scaling 校准（推断：课程未展开）
"cross-entropy 和 MSE 差不多"	MSE 配 softmax 会让梯度里多一个 $\sigma'(s)$ 因子，饱和区梯度消失；CE 的梯度是线性的 $p - y$
"初始化 loss 应该是 0"	应该是 $\log C$。为 0 说明标签泄漏

误区	纠正
"`exp` 溢出只是精度问题"	会直接产生 `nan` 并污染整个 batch 的梯度

第 6 章 专题：三种分类器的横向对比

6.1 一张表看完

维度	1-NN / kNN	线性 SVM	Softmax（多类逻辑回归）
参数	无（存训练集）	W, b	W, b
训练代价	$O(1)$	$O(NDC)$ / epoch	同左
推理代价	$O(ND)$	$O(DC)$	$O(DC)$
损失	无（投票）	$\sum_j \max(0, s_j - s_y + \Delta)$	$-s_y + \log \sum_j e^{s_j}$
输出	类别	分数（无界）	概率（和为 1）
梯度	无	0/1 计数的外积	$p - \text{one-hot}$
对错分点的惩罚	—	线性	指数
决策边界	高度非线性（Voronoi）	线性超平面	线性超平面
能否给"置信度"	可用邻居距离近似	否	是（但可能过自信）
是否需要正则	否	是（W 不唯一）	是
CIFAR-10 量级	~28%	~35–38%	~35–40%

6.2 三个"同一件事"的等价视角

1. **SVM 与 softmax 只差损失函数**。评分函数、正则、优化器完全一样。所以代码结构可以复用：`score → loss → grad`。
2. **两者都是"模板匹配"**。$s_c = w_c^\top x$，w_c 是第 c 类的模板；区别只在"如何惩罚错误"：SVM 只要领先 1 分就停止惩罚（hinge 的稀疏性 → 只有支持向量贡献梯度），softmax 永远想更大（密集梯度）。
3. **kNN 是线性分类器的极端**：当 $k = 1$ 且距离用内积时，kNN 退化为"最近模板"。这也解释了为什么"特征 + kNN"能逼近"特征 + 线性分类器"。

6.3 该用哪个：一个决策清单

```
需要概率输出 / 多标签 / 蒸馏?            -> softmax
数据量极小 (<100/类) 且特征已很好?       -> kNN 或线性 SVM (快、可解释)
需要 margin 鲁棒性 / 结构化预测?          -> 结构化 SVM (课程未讲, 推断)
输入是原始像素?                           -> 都别用, 上 CNN
需要在线学习 / 频繁增删样本?              -> kNN (无训练)
```

6.4 一个常被忽略的工程点：损失尺度

hinge loss 的量级是 $O(\Delta \cdot C)$，softmax 的量级是 $O(\log C)$ 到 $O(\text{logit 差})$。当你把 λ 从 SVM 代码搬到 softmax 代码时，**最优 λ 会差一个量级**。不要假设超参可跨损失复用，这是 A1 里最常见的"我明明调好了"翻车点。

第 7 章 正则化与优化：从"能拟合"到"能泛化"

7.1 正则化到底在做什么

总损失

$$L(W) = \underbrace{\frac{1}{N}\sum_i L_i(W)}_{\text{data loss}} + \underbrace{\lambda R(W)}_{\text{regularization}}$$

讲义的措辞很精确：data loss 要求"匹配训练数据"，regularization 要求"别匹配得太好"。三条理由：

1. **表达偏好**（express preferences）：在等价解里选一个；
2. **简化模型**（simpler model）：奥卡姆剃刀，抑制噪声拟合；
3. **改善优化**（add curvature）：加严格凸项让 Hessian 正定，问题更好解。

常见形式：

名称	形式	偏好	梯度
L2 / weight decay	$\sum_k w_k^2$	权重**摊匀**（小且分散）	$2w$
L1	$\sum_k \lvert w_k \rvert$	权重**稀疏**（很多为 0）	$\text{sign}(w)$
Elastic net	$\alpha\lVert w\rVert_1 + \beta\lVert w\rVert_2^2$	介于两者	两者相加

直觉（讲义原题）：两个解 $w_1 = (1, 0)$、$w_2 = (0.5, 0.5)$，L2 偏好 w_2（$0.25 + 0.25 = 0.5 < 1$），L1 两者等价（$1 = 1$），所以 L1 的等值线是菱形、与坐标轴相切 → 稀疏。

更广义的正则化（讲义列表）：dropout、batch normalization、stochastic depth、fractional pooling。注意 BN 被归入正则化——因为它引入了批统计噪声，起了轻微的正则效果（推断：这是被广泛接受的解释，但

BN 的主要作用仍是优化）。

7.2 优化：从随机搜索到梯度

讲义先演示了"随机搜索 W"能到 15.5% 准确率（CIFAR-10），并注明 SOTA 约 99.7%。这个数字的作用是建立"梯度有多重要"的对照。

数值梯度 vs 解析梯度：

	数值梯度	解析梯度
速度	慢（每个参数 2 次前向）	快（一次反向）
精度	近似（$O(h^2)$ 中心差分）	精确
易错性	不易错	极易错
用途	**只用于检验**	实际训练

中心差分：

$$\frac{\partial L}{\partial w} \approx \frac{L(w+h) - L(w-h)}{2h}, \qquad h \approx 10^{-5}$$

讲义用 $h = 0.0001$ 演示：$L = 1.25347$，w_1 加 h 后 $L = 1.25322$，估出梯度 -2.5。**注意**：h 太大截断误差大，太小舍入误差大；float64 下 10^{-5} 是甜点，float32 下要放大到 10^{-3} 量级（推断：float32 有效位数约 7 位）。

7.3 一阶优化器全家桶

记 $g_t = \nabla_W L(W_t)$，学习率 α。

SGD

$$W_{t+1} = W_t - \alpha g_t$$

SGD + Momentum（把梯度当"力"，累积成"速度"）

$$v_{t+1} = \rho v_t + g_t, \qquad W_{t+1} = W_t - \alpha v_{t+1}$$

ρ 常取 0.9。作用：在"峡谷"地形（一个方向陡、一个方向平）里抑制震荡、加速平坦方向。讲义引用 "Why Momentum Really Works" 作为延伸阅读。

Nesterov Momentum（先按速度走一步，再在"前瞻点"算梯度）

$$v_{t+1} = \rho v_t + \nabla L(W_t + \rho v_t), \qquad W_{t+1} = W_t - \alpha v_{t+1}$$

实践中常改写为等价形式（用 $\tilde{W} = W + \rho v$ 换元），因为框架只提供"在 W 处求梯度"的接口。

AdaGrad（按历史平方和缩放，逐参数学习率）

$$s_{t+1} = s_t + g_t^2, \qquad W_{t+1} = W_t - \alpha \frac{g_t}{\sqrt{s_{t+1}} + \epsilon}$$

问题：s 单调增，步长单调衰减到 0，非凸深层网络里过早"停住"。

RMSProp（把累加换成指数滑动平均，"leaky AdaGrad"）

$$s_{t+1} = \beta s_t + (1-\beta) g_t^2$$

Adam（动量 + RMSProp + 偏差校正）

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t, \qquad v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t^2$$

$$\hat{m} = \frac{m_{t+1}}{1-\beta_1^{t+1}}, \qquad \hat{v} = \frac{v_{t+1}}{1-\beta_2^{t+1}}, \qquad W_{t+1} = W_t - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

论文默认 $\beta_1 = 0.9,\ \beta_2 = 0.999,\ \epsilon = 10^{-8}$（推断：这是 Adam 原论文的默认值，课程只讲结构不背数字；A1 的 `adam` 实现里 config 会给 ϵ）。

偏差校正为什么必要：$m_0 = v_0 = 0$ 时，前几步的 m_t 被 0 拉低，估计严重偏小。除以 $1-\beta^t$ 正好补偿。**这是作业判分的高频考点**：漏掉校正时，前几步的有效学习率会小一个量级，loss 曲线前 10 个 iteration 明显偏平。

优化器	优点	缺点	适用
SGD	泛化常最好，内存最省	对 LR 极敏感，需调度	CNN 经典配置（ResNet 训练标配 momentum=0.9）
Adam	几乎无需调 LR，收敛快	泛化有时略差，权重衰减语义不对（应配 AdamW）	Transformer、快速原型
L-BFGS	全 batch 下收敛极快	不适用 mini-batch（噪声破坏曲率估计）	小规模凸问题、风格迁移

讲义给的三条实践结论（原文大意）：

1. **Adam(W) 在很多情况下是好的默认选择**，甚至常用常数学习率也能跑；
2. **SGD+Momentum 可能超过 Adam**，但需要更仔细地调 LR 和调度；
3. 若能承担全 batch，再考虑二阶方法（L-BFGS），且**必须关掉所有噪声源**（dropout、数据增强的随机性）。

7.4 学习率调度

策略	公式/做法	备注
Step decay	每 30/60/90 epoch 乘 0.1	ResNet 经典配方，讲义明确举例

策略	公式/做法	备注
Cosine	$\alpha_t = \alpha_0 \cdot \frac{1}{2}(1+\cos(\pi t/T))$	平滑，现代默认
Linear	$\alpha_t = \alpha_0(1 - t/T)$	简单，常用在大 batch
Inverse sqrt	$\alpha_t = \alpha_0/\sqrt{t}$	Transformer 原论文（Vaswani 2017）
Warmup	前 ~5000 iter 从 0 线性升到 α_0	防止初期大 LR 把 loss 打爆

两条经验规则（讲义明示）：

- 初始 LR 太大会让 loss 直接爆炸；线性 warmup 是标准解药。
- **batch size 放大 N 倍，初始 LR 也放大 N 倍**（Goyal et al. 2017, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour"）。这条规则有理论边界（太大的 LR 会破坏稳定性），所以后续工作又加了 warmup + 梯度裁剪。

7.5 二阶方法为什么用不了

牛顿法：

$$W \leftarrow W - H^{-1}\nabla L$$

Hessian 有 $O(N^2)$ 个元素，求逆 $O(N^3)$，而 N 是千万到亿级。所以只能走拟牛顿（BFGS/L-BFGS，用秩 1 更新近似逆 Hessian），而 L-BFGS 在 mini-batch 噪声下失效——因为曲率估计需要确定性梯度。

7.6 常见误区

误区	纠正
"Adam 比 SGD 先进，所以一律用 Adam"	视觉里 ResNet/检测/分割的强 baseline 仍多是 SGD+momentum；Adam 在 Transformer 上更主流
"L2 正则 = weight decay"	只有纯 SGD 下等价；Adam 的自适应缩放会破坏等价性，所以有 AdamW（解耦权重衰减）
"学习率越小越稳"	太小会卡在坏区域，且浪费算力；先找"能让 loss 明显下降"的最大 LR
"loss 不降就是 LR 问题"	也可能是初始化、标签错位、BN 的 running stats、数据未归一化

第 8 章 神经网络：非线性的必要性

8.1 两层网络

$$s = W_2 \max(0, W_1 x + b_1) + b_2$$

$3072 \to 100 \to 10$。记号约定（**考试爱考**）：课程把"层数"按**权重层**数，所以上式叫 **2-layer net** / **1-hidden-layer net**；也常叫 MLP（multi-layer perceptron）或 fully-connected net。

为什么必须有非线性：若 σ 是恒等，则 $W_2(W_1 x) = (W_2 W_1)x$，整个网络退化成一个线性分类器。所以激活函数的存在不是为了"更像大脑"，而是为了保证**假设空间严格大于线性**。讲义还给了另一个视角：非线性等价于一个**可学习的特征变换** $\phi(x)$，把线性不可分的问题变成线性可分（同心圆、XOR 例子）。

容量：隐藏层越大，能表达的边界越复杂。讲义明确反对"用网络大小做正则"：

> 不要用模型规模当正则项，要用更强的正则化（dropout、BN、weight decay）。

因为小模型难优化、且容易欠拟合，而大模型 + 强正则通常更好。

8.2 激活函数对比

函数	公式	优点	缺点
Sigmoid	$1/(1+e^{-x})$	输出有界，可解释为"放电率"	饱和区梯度消失；输出非零中心
Tanh	$\tanh(x)$	零中心	仍饱和
ReLU	$\max(0, x)$	正区间不饱和、计算极快、收敛快（讲义引 Krizhevsky 2012：比 sigmoid 快约 6 倍）	输出非零中心；$x<0$ 时**死神经元**（dead ReLU）
Leaky ReLU	$\max(0.01x, x)$	负区间有梯度	多一个超参
GELU	$x\Phi(x)$	零点附近平滑，Transformer 标配	计算比 ReLU 贵；大负值仍趋 0

dead ReLU 的成因：若某神经元的 W, b 使得它对所有训练样本都输出负值，则梯度恒 0，参数永不更新。缓解：小正偏置初始化、Leaky ReLU、更小的 LR。

8.3 一个 20 行的完整训练循环

讲义强调"训练一个两层网络只需要约 20 行"，这就是 A1 `two_layer_net.ipynb` 的骨架：

```
# 伪代码：与前向/反向的接口约定完全对应作业里的 layers.py
h = np.maximum(0, X @ W1 + b1)      # (N, H)
s = h @ W2 + b2                     # (N, C)
loss, ds = softmax_loss(s, y)       # ds = (p - onehot)/N
loss += reg * (np.sum(W1*W1) + np.sum(W2*W2))

dh = ds @ W2.T                      # (N, H)
dh[h <= 0] = 0                      # ReLU 反传
dW2 = h.T @ ds + 2*reg*W2
db2 = ds.sum(axis=0)
dW1 = X.T @ dh + 2*reg*W1
db1 = dh.sum(axis=0)
```

这段 12 行代码包含了后续所有层的设计范式：**forward 存 cache，backward 用 cache 算梯度，梯度形状永远与参数形状一致**。

8.4 常见误区

误区	纠正
"层数越深越好"	没有残差/归一化时，深层网络训练误差反而更高（第 11 章 ResNet 的动机）
"参数量大 = 过拟合"	参数量大 + 强正则 + 大数据 = 更好；过拟合看的是 train/val gap，不是参数量
"ReLU 是线性的，所以多层 ReLU 还是线性"	分段线性 ≠ 线性；k 个 ReLU 单元可产生 2^k 种激活模式
"隐藏层宽度不重要"	宽度决定单层能表达多少"模板"，深度决定组合层次，两者不可互替（推断：这是经验结论，无严格定理）

第 9 章 专题：反向传播的完整推导与工程实现

9.1 为什么需要计算图

直接对 $L(W_1, W_2)$ 手推梯度有三个致命问题（讲义逐条列）：矩阵微积分极其繁琐、换一个损失（hinge → softmax）就要全部重推、复杂模型（检测/RNN）根本推不动。

计算图（computational graph） 把 f 拆成原子操作，每个节点只知道"局部梯度"，反向时用链式法则相乘。这带来三个工程好处：

1. **模块化**：每个层独立实现 `forward` / `backward`；
2. **复用**：同一套反向代码适配任意拓扑；
3. **可检验**：每个节点可以单独做数值梯度检验。

9.2 链式法则的两种写法

标量形式：

$$\frac{\partial L}{\partial x} = \sum_i \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial x}$$

上游梯度（upstream gradient） 记作 $\frac{\partial L}{\partial y}$，**局部梯度（local gradient）** 记作 $\frac{\partial y}{\partial x}$，则

$$\boxed{\frac{\partial L}{\partial x} = \text{upstream} \times \text{local}}$$

这句话是整章的核心。所有"门单元"（gate）的设计都是：**给定上游梯度，如何用 forward 时缓存的值快速算出对每个输入的梯度**。

9.3 四种门的模式

门	forward	backward	记忆点
`+`	$z = x + y$	$\mathrm{d}x = \mathrm{d}z,\ \mathrm{d}y = \mathrm{d}z$	加法**分发**梯度（梯度路由器）
`*`	$z = xy$	$\mathrm{d}x = y\,\mathrm{d}z,\ \mathrm{d}y = x\,\mathrm{d}z$	乘法**交换**数值
`max`	$z = \max(x, y)$	梯度只走"赢家"	类似 ReLU / max pooling 的路由
`copy`	$x \to$ 两路	两路梯度**相加**	分支处必须累加

讲义用 $f(x, y, z) = (x + y)z$，$x = -2, y = 5, z = -4$ 演示：$q = x + y = 3$，$f = qz = -12$；$\mathrm{d}f/\mathrm{d}q = z = -4$，$\mathrm{d}q/\mathrm{d}x = 1$，故 $\mathrm{d}f/\mathrm{d}x = -4$；同理 $\mathrm{d}f/\mathrm{d}y = -4$，$\mathrm{d}f/\mathrm{d}z = q = 3$。

Sigmoid 的优雅之处：$\sigma(x) = \frac{1}{1+e^{-x}}$，其导数 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$。所以反向只需 $\mathrm{d}x = \mathrm{d}z \cdot (1 - \sigma) \cdot \sigma$，而 σ 在 forward 时已算出——**一个节点，一次减法一次乘法**。这也是"用 sigmoid 而非 exp 组合"的工程理由。

9.4 向量化：雅可比视角

若 $y = f(x)$，$x \in \mathbb{R}^n$，$y \in \mathbb{R}^m$，则局部梯度是雅可比矩阵 $J \in \mathbb{R}^{m \times n}$，$J_{ij} = \partial y_i / \partial x_j$。链式法则是**雅可比乘向量**（VJP）：

$$\mathrm{d}x = J^\top \mathrm{d}y$$

深度网络里 n, m 都是百万级，显式构造 J 不可能（10^{12} 元素）。所以每个层都被设计成**从不显式构造雅可比**，只实现 VJP。这就是为什么框架里叫 `backward` 而不是 `jacobian`。

形状法则（写作业最实用的检查）：若 `forward` 把 `(N, D)` 变成 `(N, M)`，则 `backward` 必须把 `(N, M)` 的 `dout` 变回 `(N, D)` 的 `dx`，且 `dW` 的形状与 `W` 完全一致。**形状不对一定是转置写错了，先修形状再看数值。**

9.5 三个必须能手推的反向传播

(a) Affine：$y = xW + b$

$$\mathrm{d}x = \mathrm{d}y W^\top, \qquad \mathrm{d}W = x^\top \mathrm{d}y, \qquad \mathrm{d}b = \sum_n \mathrm{d}y_{n,:}$$

b 的梯度要在 batch 维上求和（广播的反向 = 归约）。

(b) Softmax + 交叉熵（融合）

设 $p = \mathrm{softmax}(s)$，$L = -\log p_y$，则

$$\frac{\partial L}{\partial s} = p - e_y$$

注意：如果先单独实现 softmax 的反向（雅可比是 $C \times C$），再乘 CE 的梯度，结果一样但慢 C 倍且数值不稳。作业里 `softmax_loss` 要求**直接返回 (loss, dscores)**，就是为了逼你写融合版本。

(c) BatchNorm

forward（训练）：

$$\mu = \frac{1}{N}\sum_i x_i, \quad \sigma^2 = \frac{1}{N}\sum_i (x_i - \mu)^2, \quad \hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

backward（对单个特征维，N 是 batch）：

$$\mathrm{d}\hat{x} = \mathrm{d}y\,\gamma, \qquad \mathrm{d}\sigma^2 = \sum_i \mathrm{d}\hat{x}_i (x_i - \mu) \cdot \left(-\tfrac{1}{2}\right)(\sigma^2 + \epsilon)^{-3/2}$$

$$\mathrm{d}\mu = \sum_i \mathrm{d}\hat{x}_i \cdot \frac{-1}{\sqrt{\sigma^2 + \epsilon}} + \mathrm{d}\sigma^2 \cdot \frac{-2\sum_i (x_i - \mu)}{N}$$

$$\mathrm{d}x_i = \frac{\mathrm{d}\hat{x}_i}{\sqrt{\sigma^2 + \epsilon}} + \mathrm{d}\sigma^2 \frac{2(x_i - \mu)}{N} + \frac{\mathrm{d}\mu}{N}$$

化简后的"闭式"（作业里 `batchnorm_backward_alt` 要求的形式，N 为该维样本数）：

$$\mathrm{d}x_i = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} \left(\mathrm{d}y_i - \frac{1}{N}\sum_j \mathrm{d}y_j - \hat{x}_i \cdot \frac{1}{N}\sum_j \mathrm{d}y_j \hat{x}_j \right)$$

这个式子把三次归约压成两次，实际更快。**它是 A2 第一个作业题的核心**，也是"为什么 BN 反向难写"的答案：μ, σ^2 都是 x 的函数，梯度必须穿过这两条路径。

9.6 数值梯度检验

```
def rel_error(x, y):
    return np.max(np.abs(x - y) / np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

判读标准（经验，非官方）：

rel_error	含义
< 1e-7	基本正确（float64）
1e-7 ~ 1e-4	可接受，通常是不可导点或数值误差
> 1e-4	有 bug
~1	完全错（常见：忘了除以 N、符号反了、转置错）

五个让检验失败的陷阱（讲义与作业都踩过）：

1. `h` **太大或太小**：10^{-5} 是 float64 的甜点；10^{-10} 会被舍入误差淹没。
2. **ReLU / max 的折点**：样本恰好落在 0 附近时数值梯度与解析梯度必然不一致，要多采几个随机点。
3. **正则项**：检验时把 `reg` 设为 0，或确保两边都含正则。
4. **随机性**：dropout / BN 的 train 模式会让两次前向结果不同，检验时必须固定随机种子或切到 test 模式。
5. **数值范围**：loss 太小（如 10^{-8}）时相对误差没有意义，应放大 loss 或改用绝对误差。

9.8 常见误区

误区	纠正
"反向传播是另一种学习算法"	它只是**求梯度的方法**；学习算法是 SGD/Adam
"梯度检验通过就万事大吉"	检验只覆盖被采样的点；形状对但语义错（如 BN 用了 test 模式）仍可能通过
"`dx` 和 `dW` 可以随便顺序返回"	接口约定必须严格遵守，否则 `layer_utils` 串起来会静默出错
"数值梯度可以用来训练"	参数量 10^6 时一次梯度要 2×10^6 次前向，完全不可行

第 10 章 卷积神经网络：把图像结构写进模型

10.1 动机：全连接层丢掉了什么

把 $32 \times 32 \times 3$ 拉平成 3072 维后，**空间邻接关系被彻底破坏**：第 $(1,1)$ 个像素和第 $(1,2)$ 个像素在向量里相邻，但第 $(1,32)$ 和第 $(2,1)$ 也相邻——模型无法区分"左右相邻"和"换行相邻"。更严重的是参数量：一层 3072×100 就是 30 万参数，而图像分辨率翻倍参数量翻 4 倍。

卷积用三个约束解决这个问题：

约束	效果
局部连接（local connectivity）	每个输出只看 $k \times k$ 邻域
权值共享（weight sharing）	同一滤波器扫全图，参数与图像大小无关
多通道（channels）	每个滤波器输出一张 activation map，K 个滤波器输出 K 张

10.2 卷积的精确计算

输入 $x \in \mathbb{R}^{C_{in} \times H \times W}$，滤波器 $w \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}$，偏置 $b \in \mathbb{R}^{C_{out}}$：

$$y_{o,i,j} = b_o + \sum_{c=1}^{C_{in}} \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} w_{o,c,u,v}\, x_{c,\, i \cdot S+u-P,\, j \cdot S+v-P}$$

输出尺寸（最该背下来的公式）：

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

讲义的标准例子：$32 \times 32 \times 3$ 输入，6 个 5×5 滤波器，stride 1，pad 0 → $28 \times 28 \times 6$，参数量 $6 \times (3 \times 5 \times 5 + 1) = 456$。

"same" padding 的推导：要 $H_{out} = H$ 且 $S = 1$，则 $2P = K - 1$，即 $P = (K-1)/2$。所以 3×3 配 pad 1、5×5 配 pad 2、7×7 配 pad 3。**偶数核（如 4×4）无法做 same padding**，这是工程上偏好奇数核的原因之一。

1×1 卷积不是"什么都不做"：它在通道维做线性组合，作用是**跨通道降维/升维**，参数量 $C_{in}C_{out}$。GoogLeNet 的 inception 模块和 ResNet 的 bottleneck 都靠它省算力。

10.3 池化

类型	计算	反向
Max pooling	窗口内取最大	梯度只回传给最大值位置（路由）
Average pooling	窗口内取平均	梯度均分到窗口内所有位置

类型	计算	反向
Global average pooling	整张特征图取平均 → 1 个数	梯度均分到全图

作用：**降维 + 引入局部平移不变性 + 扩大感受野**。注意池化**没有参数**，所以它不增加容量，只改变信息的空间分辨率。

讲义里 2×2/stride 2 的例子：输入 4×4，max 输出 $[[6, 8], [3, 4]]$，average 输出 $[[3.25, 5.25], [2.0, 2.5]]$。

10.4 感受野（receptive field）计算

这是理解"为什么深层网络能看见全局"的关键。逐层递推（r 感受野，j 累积 stride）：

$$r_{out} = r_{in} + (K - 1) \cdot j_{in}, \qquad j_{out} = j_{in} \cdot S$$

初始 $r_0 = 1, j_0 = 1$。例：三层 3×3 conv（stride 1）→ $r = 1 + 2 + 2 + 2 = 7$；即**三个 3×3 堆叠 = 一个 7×7**。

为什么偏好小核（VGG 的核心论点）：

方案	感受野	参数量（C 通道）	非线性层数
1 个 7×7	7	$49C^2$	1
3 个 3×3	7	$3 \times 9C^2 = 27C^2$	3

参数少 45%，非线性多 2 层。代价是内存访问次数变多（工程上靠 im2col / cuDNN 融合解决）。

10.5 im2col：把卷积变成 GEMM

朴素实现是 6 层循环（batch、out_channel、out_h、out_w、in_channel、k_h、k_w），在 numpy 里慢到不可用。`im2col` 把每个感受野展平成一行：

$$(C_{in}K_hK_w,\ H_{out}W_{out}) \implies \text{与 } (C_{out},\ C_{in}K_hK_w) \text{ 做一次矩阵乘}$$

代价是**内存放大 K_hK_w 倍**（重叠区域被复制）。A2 的 `conv_forward_naive` 用显式循环（正确性优先），`fast_layers.py` 用 Cython 的 im2col（速度优先）——两者的数值必须一致，这是作业的一个隐含检查点。

```
# im2col 的核心（A2 fast_layers 的思路）
cols = np.zeros((C_in*Kh*Kw, H_out*W_out))
for c in range(C_in):
    for i in range(Kh):
        for j in range(Kw):
            cols[c*Kh*Kw + i*Kw + j, :] = x_padded[:, c, i:i+H_out*S:S, j:j+W_out*S:S].rave
out = w.reshape(C_out, -1) @ cols + b.reshape(-1, 1)
```

10.6 一个 3 层卷积网络的完整前向

讲义/A2 的 `ThreeLayerConvNet`（Conv–ReLU–Pool 重复三次）：

层	输出形状	参数
conv 3×3, 32, pad 1	(32, 32, 32)	$32 \times (3 \cdot 9 + 1) = 896$
max pool 2×2, s2	(32, 16, 16)	0
conv 3×3, 64, pad 1	(64, 16, 16)	$64 \times (32 \cdot 9 + 1) = 18496$
max pool 2×2, s2	(64, 8, 8)	0
conv 3×3, 128, pad 1	(128, 8, 8)	$128 \times (64 \cdot 9 + 1) = 73856$
max pool 2×2, s2	(128, 4, 4)	0
affine → 10	(10,)	$128 \cdot 16 \cdot 10 + 10 = 20490$

合计约 1.0×10^5 参数，CIFAR–10 上 10 epoch 内可到 ~60%，远高于线性分类器的 38%。

10.7 常见误区

误区	纠正
"卷积核越大越好"	大核参数平方增长；现代设计用堆叠小核或深度可分离卷积（推断：MobileNet 路线，课程未展开）
"padding 只是为了尺寸好看"	它决定边界像素被处理的次数；`same` 让每层输出尺寸可控，是残差连接的前提
"池化层有参数"	没有。它是固定的下采样算子
"卷积是可交换的"	深度学习里的"卷积"其实是**互相关（cross-correlation）**，不翻转核；数学卷积会翻转。因为核是学出来的，翻不翻转等价

第 11 章 专题：CNN 架构演进（LeNet → ViT）

11.1 演进主线

讲义的叙事非常清晰：**深度在增加，而每次增加深度都遇到一个新的优化障碍，架构创新就是为跨越这个障碍而生**。

模型	年份	深度	ImageNet top-5	关键创新	解决的问题
LeNet-5	1998	5 层（2 conv + 3 fc）	—（MNIST）	conv + pool + fc 的模板	首次端到端训练卷积网络
AlexNet	2012	8 层	15.3%（ILSVRC'12 冠军）	ReLU、dropout、双 GPU、数据增强	证明深 CNN 可训练
ZFNet	2013	8 层	11.7%（ILSVRC'13）	可视化指导的调参（更大的 conv1）	特征可视化反哺架构
VGG-16/19	2014	16/19 层	7.3%（ILSVRC'14）	全部 3×3 conv，通道翻倍，模块化	用"深度 + 小核"替代大核
GoogLeNet	2014	22 层	6.7%（ILSVRC'14 冠军）	Inception 模块、1×1 降维、全局平均池化	在算力预算内加宽加深
ResNet	2015	18/34/50/101/152	**3.57%**（ILSVRC'15 冠军）	残差连接、BN、bottleneck	深层网络的退化问题
DenseNet	2017	121/169/201	~3.5% 量级	密集连接（每层接前面所有层）	特征复用、梯度直达
SENet	2017	152+	2.25% 量级（ILSVRC'17）	Squeeze-and-Excitation 通道注意力	通道重要性自适应
ViT	2021	12–24+ block	88%+ top-1（大数据预训练）	patch embedding + 全局自注意力	去掉卷积归纳偏置，靠数据规模取胜

> 数字来源分三档：**讲义页明确印出**的是 ZFNet 11.7%、VGG 7.3%、ResNet 3.57%、GoogLeNet 22 层、ResNet 深度集合 18/34/50/101/152；**公开赛果但我未在讲义页上见到**的是 AlexNet 15.3%、GoogLeNet 6.7%；DenseNet/SENet/ViT 的精度我标为"量级"（见 28.6）。**不要把这些当成精确引用**。

11.2 AlexNet（2012）：三件事让它成为分水岭

1. **ReLU 替代 sigmoid**：正区间不饱和，训练快约 6 倍（讲义引 Krizhevsky 2012）；
2. **Dropout（p=0.5）** 放在两个 4096 维 fc 层之间；**工程**：两块 GTX 580（3GB）上做模型并行。

结构：`conv(11×11,96,s4) – pool – conv(5×5,256) – pool – conv3×3×3 – pool – fc4096 – fc4096 – fc1000`。参数约 60M（推断：这是公开常识，讲义未给数字）。

11.3 VGG（2014）：把"深度"变成可复制的配方

设计规则极简：

- 只用 3×3 conv（stride 1, pad 1）和 2×2 max pool（stride 2）；
- 每个 stage 后通道翻倍（64→128→256→512→512）；
- 三个 fc 层（4096–4096–1000）。

代价：参数量 ~138M（推断：公开常识），其中**第一个 fc 层就占 102M**（$512 \times 7 \times 7 \times 4096$）。这解释了为什么后续架构都用全局平均池化替掉 fc——**全连接层是参数和显存的黑洞**。

11.4 GoogLeNet / Inception（2014）：宽度上的并行

Inception 模块在同一层里并行做 1×1、3×3、5×5 conv 和 3×3 max pool，然后沿通道拼接。为了让 5×5 不爆炸，在每个分支前加 1×1 conv 降维（"bottleneck"）。

三个收益：

1. **多尺度**：不同大小的核捕捉不同尺度的模式；
2. **算力可控**：1×1 降维把 5×5 的输入通道压到 16–32；
3. **无 fc**：最后用 global average pooling + 1 个线性层，参数从 138M 降到 ~5M（推断：GoogLeNet 约 5M 参数是公开常识）。

11.5 ResNet（2015）：这一章最重要的内容

现象（讲义原文逻辑）：把 20 层和 56 层的"plain"网络对比，56 层的**训练误差**和测试误差都更差。这不是过拟合（过拟合是训练误差低、测试误差高），而是**优化困难**——更深的模型至少应该能学出"前 20 层照抄 + 后面全做恒等"，但 SGD 找不到这个解。

假设：问题出在优化，不在容量。

解法：让网络拟合残差 $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{H}(\boldsymbol{x}) - \boldsymbol{x}$，而不是直接拟合 $\boldsymbol{H}(\boldsymbol{x})$：

$$\boldsymbol{H}(\boldsymbol{x}) = \boldsymbol{F}(\boldsymbol{x}) + \boldsymbol{x}$$

若最优映射接近恒等，$F(x) \to 0$ 比 $H(x) \to x$ 容易得多（把权重压到 0 比学出恒等映射容易）。

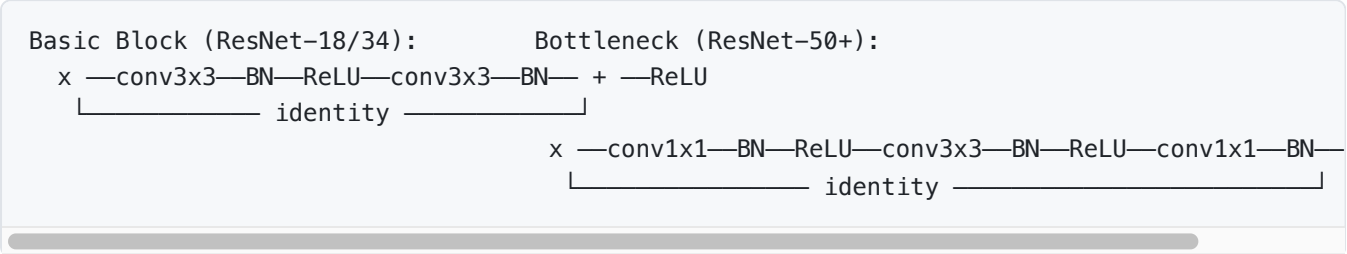
反向传播视角（这是"为什么梯度能传到底"的答案）：

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H}\left(1 + \frac{\partial F}{\partial x}\right)$$

那个 $+1$ 是**恒等路径**，保证梯度至少能以原样回传，不会因为 $\partial F/\partial x$ 很小而消失。这条"梯度高速公路"是 ResNet 能堆到 152 层甚至 1000 层的根本原因。

架构细节（讲义逐条列）：

- 每个残差块两个 3×3 conv；
- 周期性把通道数翻倍、空间下采样（stride 2），此时 shortcut 也要做 1×1 conv + stride 2 来对齐维度；
- 开头一个 7×7 conv（stem）+ max pool；
- 深度可选 18/34/50/101/152；50 层以上用 bottleneck（1×1 降维 → 3×3 → 1×1 升维）。



11.6 ViT（2021）：把归纳偏置拿掉

把图像切成 16×16 的 patch，每个 patch 线性投影成 D 维向量（$16 \times 16 \times 3 = 768 \to D$），加位置编码，送入标准 Transformer encoder，最后取 [CLS] token 或全局平均池化做分类。

关键取舍：

	CNN	ViT
归纳偏置	局部性 + 平移等变（强）	几乎没有（只有 patch 内的局部性）
数据需求	中等	**大**（ImageNet-1k 上不如同规模 CNN，JFT-300M 上反超）
计算复杂度	$O(HW \cdot K^2)$	$O((HW/P^2)^2)$，与 patch 数的平方成正比
高分辨率	直接可用	位置编码要插值

课程在第 8 讲（Attention）与第 9 讲（ViT）两次讲它，说明它是当季的重点。**（推断）** ViT 之所以能 work，是因为大规模预训练提供了 CNN 用架构先验省下来的那些样本效率。

11.7 架构选择的工程建议

```
边缘设备 / 低延迟?            -> MobileNet 系（深度可分离卷积）
通用 backbone / 迁移学习?     -> ResNet-50 仍是性价比基准
需要最强精度且数据充足?       -> ViT / ConvNeXt 系 + 大规模预训练
极小数据集 (<1万)?            -> 冻结预训练 backbone，只训线性头
需要多尺度特征（检测/分割）? -> ResNet + FPN
```

11.8 常见误区

误区	纠正
"ResNet 的 shortcut 是为了解决梯度消失"	原始动机是**退化问题（optimization）**；梯度高速公路是后来的解释，两者都对但不是同一个论点
"VGG 比 AlexNet 深所以更好"	VGG 的收益主要来自小核堆叠 + 更深 + 更强增强；参数多 3 倍，推理更慢
"Inception 和 ResNet 是对立的"	Inception-v4 就是 "ResNet + Inception"（讲义明确提到）
"ViT 完全没有卷积"	现代 ViT 常用 conv stem 或 hybrid（CNN 提特征 + Transformer 建模），纯 patchify 只是原始版本

第 12 章 训练技巧 I：初始化、归一化、正则化、数据增强

12.1 初始化：为什么 std 不能随便设

讲义的实验最有说服力：6 层网络、隐藏层 4096 维、用 $\mathcal{N}(0, 0.01^2)$ 初始化，**深层激活全部趋 0**；把 std 改成 0.05，**激活迅速爆炸**。所以初始化必须与层宽匹配。

推导（方差传播）。设 $y = \sum_{i=1}^{D_{in}} w_i x_i$，$w_i \sim \mathcal{N}(0, \sigma_w^2)$，$x_i$ 独立且方差 σ_x^2：

$$\mathrm{Var}(y) = D_{in}\,\sigma_w^2\,\sigma_x^2$$

要让 $\mathrm{Var}(y) = \mathrm{Var}(x)$，需 $\sigma_w^2 = 1/D_{in}$ —— 这就是 **Xavier 初始化**（Glorot & Bengio 2010）。它同时考虑前向和反向，取调和平均：

$$\sigma_w^2 = \frac{2}{D_{in} + D_{out}}$$

ReLU 的修正：ReLU 把一半激活置零，等效方差减半，所以要把分子翻倍：

$$\sigma_w^2 = \frac{2}{D_{in}} \quad \Longrightarrow \quad \mathrm{std} = \sqrt{2/D_{in}}$$

这就是讲义给的 **Kaiming / MSRA 初始化**（He et al. 2015, "Delving Deep into Rectifiers"）。配 ReLU 时激活在每一层都"刚好"保持尺度。

```
# A1/A2 里 FullyConnectedNet 的初始化分支
if weight_scale == 'he':
    W = np.random.randn(fan_in, fan_out) * np.sqrt(2.0 / fan_in)
elif weight_scale == 'xavier':
    W = np.random.randn(fan_in, fan_out) * np.sqrt(2.0 / (fan_in + fan_out))
```

实践补充（推断，课程未展开）：Transformer 常用 $\mathcal{N}(0, 0.02)$ 配 pre-LN；残差分支输出层做零初始化以保证初期接近恒等；偏置初始化为 0（ReLU 可给小正值避免 dead ReLU）。

12.2 数据预处理

操作	公式	说明
减均值	$x \leftarrow x - \mu$	μ 是**训练集**逐通道均值（3 个数）
除标准差	$x \leftarrow x/\sigma$	逐通道 std，现代模型几乎都做
PCA 白化	$x \leftarrow \Sigma^{-1/2}(x-\mu)$	课程提到但少用（推断：会放大噪声）

关键纪律：μ, σ 只能从**训练集**算，然后应用到 val/test。用全量数据算均值是数据泄漏（虽然影响通常小，但面试会问）。

12.3 Batch Normalization

动机：深层网络里，前面层参数一变，后面层的输入分布就变（internal covariate shift），导致必须用很小的 LR。BN 强行把每层输入拉回标准分布。

forward（训练），对一个 mini-batch 内同一特征维：

$$\mu_B = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \sigma_B^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta$$

γ, β 是**可学习**的（每维一组），让网络能恢复表达能力；ϵ 通常 10^{-5}。

forward（测试）：用**训练时累积的 running mean/var**（指数滑动平均，momentum 常 0.9）：

$$\mu_{\text{test}} = \rho\, \mu_{\text{test}} + (1-\rho)\mu_B$$

为什么测试不能用 batch 统计：推理时 batch 可能只有 1 张图，统计量无意义；而且我们希望模型对同一输入给出确定性输出。

放在哪：`affine → BN → ReLU`（conv 后、激活前）。这是原论文与讲义的顺序。

BN 的四个已知副作用：

1. 对 batch size 敏感：batch 太小（<8）时统计噪声大，性能掉；
2. 训练/测试行为不一致，是大量"线上精度掉点"的根源；
3. 与 dropout 同用时方差不匹配（推断：有 "BN + dropout 冲突" 的经验说法）；
4. 分布式训练时 BN 的统计量是**跨卡**的，需要 SyncBN（推断：课程未展开）。

变体（讲义列了归一化层家族，A2 要求实现 spatial BN 与 group norm）：

归一化	统计维度	适用
Batch Norm	(N, H, W)，对每个通道	CNN 默认，batch 大时最好
Layer Norm	(C, H, W)，对每个样本	Transformer / RNN / 小 batch
Instance Norm	(H, W)，对每个样本每个通道	风格迁移
Group Norm	通道分组后 $(C/G, H, W)$	检测/分割（batch 小），Video

12.4 Dropout

训练：以概率 p 随机把神经元置零。**测试**：全部激活，输出乘 p（或用 inverted dropout：训练时除以 $1-p$，测试什么都不做）。

讲义的两种解释：

1. **防止共适应（co-adaptation）**：不能让某个特征依赖另一个特征一定存在；
2. **隐式集成**：每个 mask 是一个子网络，4096 个单元有 $2^{4096} \approx 10^{1233}$ 种 mask（讲义原文，并调侃"宇宙只有 10^{82} 个原子"）。

inverted dropout 的实现（A1/A2 的 `dropout_forward` 要求）：

```
if mode == 'train':
    mask = (np.random.rand(*x.shape) >= p) / (1 - p)   # 注意除以 (1-p)
    out = x * mask
    cache = (dropout_param, mask)
else:
    out = x            # 测试时不做任何缩放
    cache = None
```

为什么除以 $1-p$：保持期望不变。$E[\text{mask}] = (1-p) \cdot \frac{1}{1-p} = 1$。这样测试时无需缩放，训练/测试行为一致，是框架的默认做法。

12.5 数据增强

讲义给出的配方（ImageNet 标准）：

增强	训练	测试
随机裁剪 + 缩放	短边随机取 $L \in [256, 480]$，再随机裁 224×224	固定 5 个尺度 $\{224, 256, 384, 480, 640\}$，每个尺度 10 个 crop（4 角 + 中心，含翻转），平均预测
水平翻转	$p = 0.5$	不翻转（或对翻转结果取平均）
颜色抖动	随机对比度/亮度	无
Cutout	随机挖一个方块置零	用完整图

Cutout 的适用范围（讲义明确）：在小数据集（CIFAR）上很有效，**在大数据集（ImageNet）上不常用**。原因是大数据本身已提供足够正则，Cutout 反而丢信息。

测试时增强（TTA） 的本质是"用 K 次前向换精度"，代价是 K 倍推理时间。工业界常用 2–10 个 view。

12.6 一个"正则化配方"的对照表

症状	优先加什么	不要做什么
train acc 高、val acc 低（过拟合）	数据增强 → weight decay → dropout → 早停	盲目加层
两者都低（欠拟合）	更大模型 → 更长训练 → 更高 LR	加 dropout（会更差）
训练不稳定 / loss 尖峰	BN/LN → 梯度裁剪 → LR warmup	加大 batch
小 batch 场景	GroupNorm / LayerNorm 替代 BN	硬用 BN

12.7 常见误区

误区	纠正
"BN 让网络对初始化不敏感，所以初始化不重要"	BN 缓解但不消除；残差分支的初始化仍决定能否训练
"dropout 在测试时也要按比例丢弃"	测试必须用全部神经元；inverted dropout 已在训练时缩放
"BN 的 running stats 可以省掉"	省掉会导致推理时用当前 batch 统计，batch=1 时输出随机
"数据增强只是防止过拟合"	它还编码了**不变性先验**（翻转后仍是猫），这是 CNN 泛化的核心来源之一

第 13 章 训练技巧 II：超参搜索与调试流程

13.1 六步法（讲义原文顺序）

步	做什么	判据
1	**检查初始 loss**	应 $\approx \log C$（softmax）或 $\approx C-1$（SVM，margin 全违反时）
2	**过拟合一个小样本**	取 5–10 张图，关正则，loss 应到 0（准确率 100%）
3	**找能让 loss 下降的 LR**	用全部数据、小 weight decay，试 $10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}$，看 ~100 iter 内是否显著下降
4	**粗网格**	训练 1–5 epoch
5	**细化网格**	训练更久
6	**看曲线**	见 13.3
7	回到第 5 步	迭代

第 2 步是最有价值的调试手段：**如果模型连 5 张图都过拟合不了，一定是代码 bug**（梯度错、标签错、层接错），而不是超参问题。

13.2 随机搜索 > 网格搜索

Bergstra & Bengio (2012)：网格搜索在"不重要的维度"上浪费大量试验；随机搜索让每个维度都被充分采样。极端情形下（只有一个维度重要），网格搜索跑了 K 次却只试了 $K^{1/d}$ 个不同值，随机搜索试了 K 个。实操：先在对数尺度上随机采样 LR（10^{-5} 到 10^{-1}），weight decay 同理；batch size 用 2 的幂；dropout 在 $[0, 0.5]$ 均匀采。

13.3 曲线诊断表

曲线形态	诊断	处方
train/val 都在降，但很慢	LR 太小 或 欠拟合	加大 LR、加容量、训练更久
train 降、val 平	过拟合	加正则/增强、早停
train 降、val 升	明确过拟合	同上，且考虑减容量
train 和 val 都不动	梯度消失 / 标签错 / LR 太小	查 BN、查初始化、过拟合小样本
loss 出现 `nan`	LR 太大 / 除零 / log(0)	降 LR、加 eps、梯度裁剪

曲线形态	诊断	处方
前几十 iter loss 平	初始化差 / 缺 warmup / 偏差校正漏了	换 He init、加 warmup、修 Adam

13.4 一个可直接套用的训练节奏

```
1) 过拟合 10 张图，确认代码正确（loss -> 0）
2) 找 LR：1e-4 起，loss 不降则 x3，爆炸则 /3
3) 加正则、跑长训练；每 5 epoch 同时打印 train/val acc
```

13.5 常见误区

误区	纠正
"先把所有超参一起调"	先固定架构，只调 LR；LR 对了，其他才有意义
"val 上反复试到满意再报 test"	每看一次 test 都是一次选择偏差；test 只在最后用一次
"loss 曲线平滑才健康"	有小幅震荡是正常的（mini-batch 噪声）；持续单调下降反而可能 batch 太大
"early stopping 不算正则"	它等价于限制参数范数，是最便宜的正则

第 14 章 迁移学习：小数据下的唯一正解

14.1 动机

讲义的前提很直接：**如果你手上的数据 < ~1M 张，别从零训**。步骤是

1. 找一个**大数据、相似领域**的数据集，训一个大模型；
2. 把这个模型迁移到你的数据集。

14.2 四种情形（讲义的四象限）

	与预训练数据很相似	与预训练数据很不同
数据很少	只训最后一层线性分类器（冻结其余）	从中间层提特征 + 线性分类器；或换更强的预训练模型
数据较多	微调全部层（小 LR）	微调全部层，或从零训练

讲义的补充：数据越多，越应该**解冻更多层**；学习率应设得很小（10^{-4} 量级），因为预训练权重已接近好的解。

14.3 三种落地方式

```
# 方式 1：特征提取（最省，冻结 backbone）
feat = backbone(x)                    # no_grad
logits = nn.Linear(feat_dim, C)(feat)

# 方式 2：微调（finetune）
for p in backbone.parameters(): p.requires_grad = True
optimizer = SGD([{'params': backbone.parameters(), 'lr': 1e-4},
                 {'params': head.parameters(),     'lr': 1e-3}])  # 分层 LR

# 方式 3：替换分类头 + 差分学习率（工业界最常用）
```

BN 层在微调时的特殊处理（推断，工程经验）：小 batch 微调时通常**冻结 BN 的 running stats**（`bn.eval()`），否则统计量会被小 batch 带偏，精度掉得莫名其妙。

14.4 为什么迁移学习有效

- **低层特征通用**：Gabor 类边缘/颜色滤波器在不同数据集上几乎一样（讲义用 "L2 nearest neighbors in feature space" 证明 fc7 特征有语义）；
- **高层特征任务相关**：所以换头、微调高层是标准做法；
- **有效样本量放大**：预训练相当于给了模型一个强先验，等价于增加了数据。

14.5 模型动物园

讲义点名两个来源：PyTorch 的 `torchvision.models`（`github.com/pytorch/vision`）与 HuggingFace 的 `timm`（`github.com/huggingface/pytorch-image-models`）。工程上还要注意：**预处理必须与预训练时一致**（同样的 mean/std、同样的 resize/crop），否则精度会掉几个点——这是最高频的"迁移学习翻车"原因。

14.6 常见误区

误区	纠正
"微调一定比冻结好"	数据少时微调会过拟合/灾难性遗忘；先试冻结
"随便换个 backbone 就能提升"	预处理、输入分辨率、归一化不一致会让提升消失

误区	纠正
"预训练模型越大越好"	推理成本、显存、延迟都要算；边缘端常用 MobileNet/Distilled 模型

第 15 章 循环网络与序列视觉任务

15.1 为什么视觉需要序列模型

视觉任务不只有"一图一标签"：

任务	输入	输出
图像描述（captioning）	1 张图	T 个词
动作识别	T 帧	1 个动作类
视频描述	T 帧	T' 个词
逐帧分类	T 帧	T 个标签

15.2 Vanilla RNN

$$h_t = \tanh\left(W_{hh} h_{t-1} + W_{xh} x_t + b_h\right), \qquad y_t = W_{hy} h_t + b_y$$

关键点：所有时间步共享同一组 W（和 CNN 的权值共享同源）。计算图是"同一函数沿时间展开"，反向就是 BPTT（backpropagation through time）——把展开图当普通前馈网络反传，梯度沿时间累加。

梯度爆炸/消失：$\partial h_t / \partial h_{t-1}$ 连乘 T 次，若谱半径 < 1 则指数衰减，> 1 则爆炸。处方：梯度裁剪（clip 到 $[-5, 5]$）、LSTM/GRU、truncated BPTT（只回传 k 步）。

15.3 LSTM 与 GRU

LSTM 用**门控 + 细胞状态**把"记忆"从"隐状态"里分离：

$$\begin{aligned}
f_t &= \sigma(W_f [h_{t-1}, x_t] + b_f) && \text{遗忘门} \\
i_t &= \sigma(W_i [h_{t-1}, x_t] + b_i) && \text{输入门} \\
o_t &= \sigma(W_o [h_{t-1}, x_t] + b_o) && \text{输出门} \\
\tilde{c}_t &= \tanh(W_c [h_{t-1}, x_t] + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{细胞状态更新} \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}$$

c_t 的更新是**加法**，梯度可以沿 c 直接回流（类似残差的恒等路径），这是 LSTM 缓解梯度消失的机制。GRU 把遗忘/输入门合并成"更新门"，少一组参数。

15.4 图像描述：CNN + RNN

标准架构（讲义/A2 的 `RNN_Captioning`）：

```
image -> CNN (VGG/ResNet) -> fc -> h0  (图像作为 RNN 的初始隐状态)
                                    |
"<START>" -> W_embed -> RNN -> RNN -> RNN -> ... -> "<END>"
                          |      |      |
                        "a"   "cat"  "sits"
```

训练：teacher forcing（每步喂真实的上一个词），损失是每步 softmax CE 之和（padding 位置 mask 掉）。 测试：**自由采样**——每步取 argmax（或 beam search），把输出喂回下一步，直到 `<END>` 或达到最大长度。

必须注意：训练用 teacher forcing 而测试用自回归采样，这个 train/test 不一致（exposure bias）是 captioning 的经典问题（推断：课程未展开 scheduled sampling）。

15.5 与后续的衔接

RNN 的三个问题——**不能并行**（h_t 依赖 h_{t-1}）、**长程依赖仍难**、$O(T)$ **串行计算**——正是第 16 章 Transformer 的动机。讲义在第 9 讲开篇用一张表对比了三种序列处理方式（RNN / CNN / self-attention）：

	RNN	1D CNN	Self-Attention
作用对象	有序序列	N 维网格	**向量集合**
长程依赖	理论 $O(N)$，实际难	需堆叠很多层	每层直接看全部，$O(1)$ 路径
并行	❌ 必须串行	✅	✅ 四次矩阵乘
复杂度	$O(N)$	$O(N \cdot K)$	$O(N^2)$ 计算 / $O(N)$ 内存

15.6 常见误区

误区	纠正
"LSTM 解决了梯度消失"	缓解，未根治；极长序列仍会遗忘
"captioning 的 loss 就是分类 loss"	是**每步** CE 的和/平均，且要 mask 掉 padding
"测试时也用 teacher forcing"	测试没有 ground truth，必须自回归

误区	纠正
"RNN 已被完全淘汰"	流式/低延迟场景仍有优势；状态空间模型（Mamba）是它的现代后继（推断）

第 16 章 注意力与 Transformer：视觉里的"集合运算"

16.1 从 seq2seq 的瓶颈说起

编码器把整个输入压成一个固定长度的 $c = h_T$，解码器只看 c。当 $T = 1000$ 时信息必然丢失（讲义原话："bottlenecks through fixed sized c"）。注意力（Bahdanau et al. 2015）的解法是：**解码每一步都回头看整个输入序列**。

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i), \qquad a_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \qquad c_t = \sum_i a_{t,i} h_i$$

三个动作：打分 → softmax 归一化 → 加权求和。$a_{t,i}$ 可解释为"对齐权重"，这是可视化注意力的来源。

16.2 自注意力（self-attention）

把"查询"也变成一个向量序列，让序列内部互相关注：

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

符号	形状	含义
Q	$N \times d_k$	query，我想找什么
K	$N \times d_k$	key，我提供什么索引
V	$N \times d_v$	value，我实际携带的内容
$QK^\top/\sqrt{d_k}$	$N \times N$	两两相似度（N^2 的来源）

为什么除以 $\sqrt{d_k}$：$q \cdot k$ 的方差随 d_k 线性增长，softmax 在 logits 很大时会饱和（梯度趋 0）。除以 $\sqrt{d_k}$ 把方差归一到 1。这是面试高频题。

多头：把 d 维切成 h 份，每份独立做 attention，再拼接投影。作用类似 CNN 的多通道——不同头关注不同的关系（语法、位置、颜色……）。

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O, \qquad \text{head}_i = \text{Attn}(XW_i^Q, XW_i^K, XW_i^V)$$

16.3 Transformer block 的两个版本

```
Post-LN（原论文 2017）：            Pre-LN（现代默认）：
  x = x + Attn(LN(x))                 x = x + Attn(LN(x))
  x = x + FFN(LN(x))                  x = x + FFN(LN(x))
  # LN 在残差之内                     # LN 在残差分支内、残差外
```

讲义明确讲了为什么要 Pre-LN：Post-LN 里 LN 在残差**之后**，模型"无法学出恒等映射"（因为 LN 会改尺度），深层训练不稳定；把 LN 移到子层输入侧后，残差路径是纯恒等，训练更稳（Baevski & Auli 2018）。这和 ResNet 的 $+1$ 梯度高速公路是同一个思想。

RMSNorm：去掉均值减法，只除以均方根，省一次归约，效果几乎不变（讲义提到，现代 LLM 标配）。

FFN：两层线性 + 激活，中间维度通常是 $4d$。它承担了"逐 token 的非线性变换"，占总参数的 2/3。

16.4 ViT：把图像当词序列

```
image (3, 224, 224)
  -> 切成 14x14 个 16x16 patch
  -> 每个 patch 展平 (768) 并线性投影到 D
  -> 加位置编码（可学习或正弦）
  -> 前置一个 [CLS] token
  -> L 个 Transformer block
  -> 取 [CLS] 的输出 -> 线性层 -> C 类
```

与 NLP Transformer 的唯一结构差别：输入是 patch 而不是 token，位置编码是 2D 的。其他完全一样（讲义原话："Exact same as NLP Transformer"）。

复杂度对比（N 个 patch，d 维）：

层	计算量	备注
Self-attention	$O(N^2 d)$	patch 越多越贵（$N = (H/P)^2$）
FFN	$O(Nd^2)$	与 N 线性
Conv（同感受野）	$O(HWK^2C^2)$	与分辨率线性

所以 ViT 在高分辨率下反而更贵，常用 window attention / hybrid 来缓解（推断：Swin 路线，课程未展开）。

16.5 三种序列建模的取舍（复习用）

	RNN	CNN	Self-Attention
归纳偏置	顺序性	局部性 + 平移等变	置换不变（需位置编码）
长程依赖	需多步传播	需堆叠	一步直达
并行度	低	高	高
显存	$O(N)$	$O(N)$	$O(N^2)$
适合	流式、小序列	图像、局部模式	长序列、多模态

16.6 常见误区

误区	纠正
"attention 就是加权平均"	权重来自 query–key 的**内容相关**相似度，不是固定的
"位置编码可有可无"	去掉后模型对 patch 顺序完全不敏感（置换等变），图像变成"一袋 patch"
"ViT 一定比 CNN 好"	只在**大规模预训练**后成立；小数据上 CNN 仍占优
"多头 = 多模型集成"	头之间共享输入，只是把通道分组，不是独立模型

第 17 章 专题：检测与分割

17.1 四个任务的输出空间

任务	输出	关键难点
分类	1 个标签	语义
语义分割（semantic segmentation）	$H \times W$ 的类别图	逐像素、不区分实例
目标检测（object detection）	K 个 $(x, y, w, h, \text{class})$	**数量可变**、需 NMS
实例分割（instance segmentation）	K 个 mask + 类别	逐实例的像素级输出

17.2 语义分割的三代做法

(a) **滑窗**：对每个像素取一个 patch 分类。问题：重叠区域不共享计算，慢到不可用。

(b) 全卷积（FCN, Long et al. 2015）：整个网络只有 conv，输出 $C \times H \times W$ 的 score map，再 argmax。问题：为了增大感受野必须下采样，但分割要求输出与输入同尺寸。

(c) 编码器–解码器：下采样 + 上采样。

上采样方式	做法	是否可学习
最近邻 / "bed of nails"	复制或填零	否
Max unpooling	记住 max 位置，把梯度放回原位	否
转置卷积（transposed conv / deconv）	学习的上采样，stride 控制放大倍数	是

转置卷积的本质：把 forward 的"卷积矩阵"转置。1D 例子：输入 $[x, y, z]$，核 $[a, b, c]$，stride 2 → 输出里包含 $ax,\ ay + bx,\ az + by,\ bz$ 等项（重叠处相加）。**它不是卷积的逆**，只是形状上的转置，所以会产生棋盘格伪影（checkerboard artifact，推断：这是 Odena 2016 的著名观察，课程未展开）。

U-Net（Ronneberger et al. 2015）是这一章的标配答案：

```
encoder（下采样，扩大视野，丢空间信息）
   └── skip connection（把高分辨率特征拼到 decoder）──┐
decoder（上采样，恢复分辨率）   <──────────────────────┘
```

skip connection 解决"下采样丢细节"的问题，是分割/超分/去噪的通用结构。

17.3 目标检测：两阶段 vs 单阶段

(a) R-CNN（2014）：选择性搜索（selective search）产生 ~2000 个候选框 → 每个框 warp 到 224×224 → 各自过 CNN → SVM 分类 + 边框回归。问题：2000 次 CNN 前向，极慢。

(b) Fast R-CNN（2015）：整图只过一次 CNN，用 **RoI pooling** 在特征图上取每个候选框的特征。速度大幅提升，但候选框仍来自外部算法。

(c) Faster R-CNN（2015）：用 **RPN（Region Proposal Network）** 替代选择性搜索。在特征图每个位置放 K 个 anchor（不同尺度/长宽比），预测"是否有物体" + "相对 anchor 的偏移 (dx, dy, dw, dh)"，按 objectness 排序取 top ~300 作为 proposals。**这是第一个端到端可训练的两阶段检测器。**

(d) YOLO（2016）/ SSD（2016）/ RetinaNet（2017）：单阶段。把图分成 $S \times S$ 网格，每格预测 B 个框的 $(x, y, w, h, \text{conf})$ 和 C 个类别分数，输出张量 $S \times S \times (5B + C)$（讲义给的正是这个形式，$B = 3$）。

	两阶段	单阶段
精度	通常更高（尤其小物体）	略低
速度	慢	**快**（实时）
代表	R-CNN / Fast / Faster / Mask R-CNN	YOLO / SSD / RetinaNet

	两阶段	单阶段
核心问题	候选框质量	**正负样本极度不平衡**（RetinaNet 用 focal loss 解决）

(e) DETR（2020）：用 Transformer 直接输出一个**集合**的框，无需 anchor、无需 NMS。用匈牙利匹配做二分图匹配分配预测与 GT。讲义在第 9 讲末尾提到了它（"no anchors, no regression of box transforms"）。

17.4 Mask R-CNN：把检测扩展到实例分割

在 Faster R-CNN 的每个 RoI 上**并行**加一个小的 mask 分支（14×14 的 FCN，输出 K 个 28×28 的 mask），与分类/回归分支并列训练。

两个关键设计：

1. **RoIAlign 替代 RoI pooling**：用双线性插值代替量化取整，解决 mask 与特征图**像素不对齐**的问题（这是 Mask R-CNN 相对前作最大的精度提升点）；
2. **mask 与类别解耦**：每个类独立预测一个 mask，避免类间竞争。

17.5 指标：IoU 与 mAP

IoU（Intersection over Union）：

$$\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}$$

AP 的计算（单类）：

1. 把所有预测按置信度降序排列；
2. 逐个累加 TP/FP，算 precision 与 recall；
3. 画 $p(r)$ 曲线，AP = 曲线下面积（VOC2010 前用 11 点插值，之后用全点插值）；
4. **mAP** = 所有类 AP 的平均。

$$\text{AP} = \int_0^1 p(r)\,dr \approx \sum_k (r_{k+1} - r_k)\, p_{\text{interp}}(r_{k+1})$$

COCO 的口径更细：IoU 从 0.5 到 0.95 步长 0.05 取平均（AP@[.5:.95]），并区分 small/medium/large 物体。**报数时必须写清楚是哪个口径**，否则 0.5 和 0.75 能差 10 个点以上。

NMS（非极大值抑制）：

```
def nms(boxes, scores, iou_thresh=0.5):
    order = scores.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = order[0]; keep.append(i)
        ious = compute_iou(boxes[i], boxes[order[1:]])
        order = order[1:][ious <= iou_thresh]   # 抑制高重叠框
    return keep
```

17.6 全景分割与评价

- **语义**：每像素一个类，不区分实例；
- **实例**：每个物体一个 mask；
- **全景（panoptic）**：stuff（天空、草地，不可数）+ thing（人、车，可数）统一，每个像素恰好一个标签。

17.7 常见误区

误区	纠正
"mAP 越高模型越好"	不同 IoU 阈值、不同数据集（VOC vs COCO）不可直接比较
"YOLO 精度低是因为网络小"	主因是单阶段的正负样本不平衡与定位精度权衡
"转置卷积是卷积的逆运算"	只是形状转置，信息不可逆；会产生棋盘伪影
"RoI pooling 和 RoIAlign 差不多"	差一个插值，Mask R-CNN 的 mask 精度提升主要来自这里
"anchor 是必须的"	DETR / FCOS 等 anchor-free 方法证明不必须

第 18 章 可视化与可解释性

18.1 五个层次的可视化

方法	看什么	怎么做
第一层滤波器	网络学到了什么基元	把 $\boldsymbol{W}$ 的 $\mathbf{3}\times\mathbf{7}\times\mathbf{7}$ 通道可视化
激活图	某个神经元对什么响应	取某通道的 feature map
显著性图（saliency）	哪些像素影响分类	$\boldsymbol{\partial s_c/\partial x}$，取绝对值、跨通道取 max

方法	看什么	怎么做
CAM / Grad-CAM	哪些**区域**支持某类	用最后卷积层的加权和
特征反演 / DeepDream	网络的"想象"	优化输入使某神经元激活最大

18.2 CAM 的推导（讲义给了完整式子）

设最后卷积层特征 $f \in \mathbb{R}^{H\times W\times K}$，全局平均池化得 $F_k = \frac{1}{HW}\sum_{h,w} f_{h,w,k}$，分类层权重 $w \in \mathbb{R}^{K\times C}$，则

$$S_c = \sum_k w_{k,c} F_k = \sum_k w_{k,c} \frac{1}{HW}\sum_{h,w} f_{h,w,k} = \frac{1}{HW}\sum_{h,w}\underbrace{\sum_k w_{k,c} f_{h,w,k}}_{\mathrm{CAM}_{h,w}}$$

所以类别 c 的 CAM 就是 $\sum_k w_{k,c} f_{h,w,k}$——**对特征图按分类权重加权求和**。上采样到原图尺寸即得热力图。

局限：要求网络结构是"conv → GAP → fc"，否则没有 w。

Grad-CAM 把 $w_{k,c}$ 换成梯度：$w_{k,c} = \frac{1}{HW}\sum_{h,w}\frac{\partial S_c}{\partial f_{h,w,k}}$，从而适用于任意架构。

18.3 对抗样本与 DeepDream

- **对抗样本**：沿 $\nabla_x L$ 走一小步（L_∞ 约束下），人眼看不出变化但分类器错得离谱。这说明模型的决策边界与人类的感知边界不一致。
- **DeepDream**：不是最小化 loss，而是**最大化**某个中间层神经元的激活，让图像里"长出"网络学到的模式（狗脸、眼睛）。迭代多次会产生迷幻效果。
- **风格迁移**：内容损失（高层特征匹配）+ 风格损失（Gram 矩阵匹配），优化一张图。Gram 矩阵 $G_{ij} = \sum_k F_{ik}F_{jk}$ 捕捉通道间的相关性，等价于"纹理统计"。

18.4 这些方法能信吗

局限	说明
显著性图不是解释	它只说明"梯度大"，不说明"因果"
CAM 有分辨率上限	受最后层 stride 限制，通常 1/32 原图
对抗鲁棒性与可解释性不是一回事	显著性好不代表模型鲁棒
双刃剑	同一套梯度方法既能解释也能攻击

18.5 常见误区

误区	纠正
"可视化好看就说明模型学到了正确的东西"	模型可能靠背景/水印/纹理捷径（shortcut）分类
"CAM 需要改网络结构"	原始 CAM 需要 GAP，Grad-CAM 不需要
"对抗样本是过拟合"	是**高维线性**的普遍性质，与过拟合无关（推断：这是 Goodfellow 2014 的线性解释）

第 19 章 视频理解：2D + 时间

19.1 问题的规模

视频是 4D 张量 $T \times 3 \times H \times W$。讲义给了一组很直观的数字：

分辨率	未压缩体积（3 字节/像素，30fps）
SD 640×480	~1.5 GB / 分钟
HD 1920×1080	~10 GB / 分钟

所以不能"逐帧全量训练"。标准做法：**训练用短片段**（如 $T = 16$、112×112、5fps → 3.2 秒、588 KB），**测试时跑多个 clip 再平均**。

19.2 五种融合策略

方法	做法	优点	缺点
单帧 CNN	每帧独立分类，测试时平均	极强 baseline	完全无运动信息
Late fusion（FC）	各帧特征拼接后过 MLP	保留高层外观	参数量随 T 增长
Late fusion（pooling）	各帧特征在时间维平均	支持任意长度	无法建模低层运动
Early fusion（3D conv）	一开始就用 3D 卷积	能学运动	计算/显存大
Two-stream	RGB 流 + 光流流，晚融合	显式建模运动	光流预计算昂贵

讲义的评价：单帧 CNN "often a very strong baseline"，这句话在工程上很重要——**先跑单帧 baseline，再决定是否值得上 3D**。

19.3 3D 卷积的代价

把 $K \times K$ 变成 $K \times K \times K$，参数量乘 K，FLOPs 乘 K。所以 3D CNN 通常配"浅层 + 小分辨率 + 短片段"。后来的解法是 (2+1)D 卷积（空间 2D + 时间 1D 分离），推断：课程未展开。

19.4 常见误区

误区	纠正
"视频分类 = 多帧图像分类平均"	动作类（"开门" vs "关门"）必须靠时序，单帧无法区分
"帧率越高越好"	训练用 5fps 足够，高帧率主要提升光流质量
"3D CNN 一定优于 2D"	在小数据集上 3D 更容易过拟合，2D + 时序池化常更强

第 20 章 大规模分布式训练

20.1 三种并行

并行	切什么	通信量	适用
数据并行（data parallel）	切数据，每个 worker 有完整模型副本	每步 all-reduce 梯度（= 参数量）	模型能放进单卡
模型并行（model parallel / tensor parallel）	切模型层内	每层激活	单层都放不下
流水并行（pipeline parallel）	切模型层间	阶段间激活	层数很多

讲义的定义很直白：数据并行 = "Copy the model to all workers, split the data"；模型并行 = "Split model across devices"。同步 vs 异步梯度更新是另一个维度：同步（每步等所有 worker）收敛稳定但受最慢者拖累；异步吞吐高但有 stale gradient。

20.2 通信的物理尺度（讲义给的 Llama3 案例）

层级	带宽
H100 卡内 HBM	3352 GB/s
同服务器 8 卡（NVLink）	900 GB/s

层级	带宽
同 pod（3072 卡）	50 GB/s
跨 pod	< 50 GB/s

Llama3–405B 的训练集群是 8 pod × 3072 = **24,576 张 H100**。这张表的教学价值在于：**通信带宽随规模断崖式下降**，所以并行策略的本质是"把通信放在带宽最高的层级"。

20.3 显存都花在哪

项	大小（每参数）
权重（fp16）	2 B
梯度（fp16）	2 B
优化器状态（Adam: fp32 m + v）	8 B
激活	与 batch × 序列长度 × 层数成正比

所以混合精度 + Adam 的"每参数 12 B"是估算显存的第一原则。三个省显存的手段：

1. **混合精度**（fp16/bf16 前向反向，fp32 主权重）；
2. **activation checkpointing**（不存中间激活，反向时重算，用算力换显存，讲义第 11 讲标题明确列出）；
3. **ZeRO / FSDP**（把优化器状态、梯度、参数分片到各卡，推断：课程讲原理但未展开实现）。

20.4 大 batch 的两个陷阱

1. **LR 要缩放**：batch × N → LR × N，但要配 warmup，否则初期发散（第 7 章）。
2. **BN 的统计量跨卡**：数据并行时每卡只看到局部 batch，BN 统计会有偏，需要 SyncBN（推断）。

20.5 常见误区

误区	纠正
"数据并行能训任意大的模型"	模型必须放得进单卡（除非配 FSDP/ZeRO）
"异步 SGD 一定更快"	吞吐高但需要调 staleness，收敛常变差
"多卡一定线性加速"	通信 + 负载不均，实测常是 70–90% 效率

第 21 章 专题：自监督学习

21.1 动机

监督学习的瓶颈是标注。自监督（self-supervised）的思路：**从数据本身构造监督信号**（pretext task），学到的表示再迁移到下游任务。

```
无标签数据 --(pretext task)--> encoder --(linear probe / finetune)--> 下游任务
```

常见 pretext：图像补全（inpainting）、旋转预测（rotation）、拼图（jigsaw）、着色（colorization）、掩码重建（MAE）。

评估协议（讲义列了五种，最重要的是前两个）：

协议	做法
Linear probe	冻结 encoder，只训一个线性分类器
Fine-tune	解冻全部，小 LR 微调
Clustering / t-SNE	看表示的可分性
迁移到检测/分割	看下游任务增益
计算效率	训练时间/显存

21.2 对比学习：SimCLR

核心思想：**同一张图的两个增强视图应该相似，不同图的视图应该不相似。**

对一个 batch 的 $2N$ 个视图，正样本对是 $(i, i+N)$，其余 $2N-2$ 个都是负样本。InfoNCE 损失（k 为正样本索引，τ 为温度）：

$$\ell_i = -\log \frac{\exp(\mathrm{sim}(z_i, z_k)/\tau)}{\sum_{j \neq i} \exp(\mathrm{sim}(z_i, z_j)/\tau)}$$

$\mathbf{sim}$ 是归一化点积（余弦相似度）。总损失对 $2N$ 个位置平均。

SimCLR 的三个关键发现（推断：来自原论文，课程讲了结论）：增强的组合至关重要（随机裁剪 + 颜色抖动最强）、需要大 batch 提供足够负样本、非线性投影头（MLP）之后接损失效果更好。

MoCo 解决"大 batch"约束：用**动量编码器** + **队列**存历史 key，从而用小 batch 也能有大量负样本。梯度只通过 query 编码器回传。

21.3 自蒸馏：DINO

学生网络预测教师网络（动量更新）的输出分布，用 **KL 散度**（而非点积）匹配，配 ViT。DINO 的副产品是 attention map 自动出现了语义分割效果（讲义展示了这点，也是 A3 里 DINO attention 可视化的来源）。DINOv2 把数据从 1M 扩到 142M，成为目前最常用的通用视觉特征之一。

21.4 掩码重建：MAE

把图像切成 patch，随机遮掉 75%，让 ViT 重建被遮住的像素。两个设计要点：

1. **非对称编码器–解码器**：编码器只看可见 patch（省 3/4 算力），轻量解码器重建；
2. **高掩码率**：75% 迫使模型学语义而不是局部插值。

与 NLP 的 BERT（遮 15% token）不同，图像的冗余度高，必须遮得更多才有难度。

21.5 一张对照表

方法	信号	损失	代表模型
生成式（重建）	重建像素	MSE	MAE、BEiT
对比式	正负样本对	InfoNCE	SimCLR、MoCo
自蒸馏	教师输出分布	KL	DINO、BYOL
pretext 分类	变换标签	CE	Rotation、Jigsaw

21.6 常见误区

误区	纠正
"自监督 = 无监督"	它有监督信号，只是**自动生成**的
"linear probe 高就说明特征好"	还要看低数据量下的表现和下游检测/分割
"负样本越多越好"	负样本质量（难度）比数量更重要；MoCo 的队列就是为了质量
"MAE 适合所有模态"	高冗余模态（图像/视频）才适合高掩码率；文本不行

第 22 章 专题：生成模型（VAE / GAN / 扩散）

22.1 分类图（讲义第 14 讲的划分）

生成模型先分**显式密度**（能算 $p(x)$：自回归、VAE）与**隐式密度**（不能算但能采样：GAN 直接采样、扩散间接采样）。VAE 优化的是可解下界（ELBO），GAN 完全放弃似然，扩散则优化一个可算的变分下界并把它简化成噪声回归。

22.2 自回归模型

把图像当像素序列，用链式法则：

$$p_\theta(x) = \prod_{i=1}^{N} p_\theta(x_i \mid x_1, \ldots, x_{i-1})$$

用 RNN/Transformer（PixelRNN/PixelCNN/Image GPT）建模。优点：似然可精确计算、训练稳定。缺点：**采样必须逐像素串行**，256×256 图要 65536 步。

22.3 VAE

引入隐变量 z，$p(x) = \int p(x|z)p(z)dz$ 不可解，于是最大化 **ELBO（证据下界）**：

$$\log p_\theta(x) \geq \mathbb{E}_{z\sim q_\phi(z|x)}\left[\log p_\theta(x|z)\right] - D_{KL}\left(q_\phi(z|x) \,\|\, p(z)\right)$$

项	作用
重建项	让解码器能从 z 还原 x
KL 项	让编码器分布靠近先验 $\mathcal{N}(0, I)$，保证 z 空间连续可采样

工程化的关键技巧是**重参数化（reparameterization）**：$z = \mu + \sigma \odot \epsilon$，$\epsilon \sim \mathcal{N}(0, I)$，让梯度能穿过采样。

VAE 的经典问题：重建模糊。因为 MSE 重建项对应高斯似然，倾向于输出"平均脸"。

22.4 GAN

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]$$

- D 想区分真假，G 想骗过 D；
- 最优 $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)}$，此时目标等价于最小化 JS 散度；
- **实践坑**：JS 散度在分布不重叠时饱和，梯度消失 → 改用非饱和损失 $-\log D(G(z))$；模式崩溃（mode collapse）；训练不稳定需要谱归一化/梯度惩罚（推断：WGAN-GP 路线）。

22.5 扩散模型（DDPM）

前向过程：逐步加高斯噪声，T 步后变成纯噪声。

$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right)$$

闭式（可直接跳到任意 t，$\alpha_t = 1-\beta_t$、$\bar{\alpha}_t = \prod_{s\le t}\alpha_s$）：

$$q(x_t|x_0) = \mathcal{N}\left(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\right)$$

反向过程：学一个网络预测噪声。

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1};\ \mu_\theta(x_t,t),\ \Sigma_\theta(x_t,t)\right)$$

训练目标（DDPM 的关键简化）：不直接优化 ELBO，而是回归噪声：

$$L_{\text{simple}} = \mathbb{E}_{t,x_0,\epsilon}\left[\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\ t\right)\right\|^2\right]$$

四步训练循环：

```
x0 = batch                                  # 干净图
t = randint(0, T)                           # 随机时间步
eps = randn_like(x0)                        # 真噪声
xt = sqrt(abar[t]) * x0 + sqrt(1-abar[t]) * eps
loss = mse(model(xt, t), eps)               # 预测噪声
```

采样：从 $x_T \sim \mathcal{N}(0, I)$ 开始，逐步去噪 T 次。慢是扩散模型的原罪，后续的 DDIM（少步采样）、latent diffusion（在 VAE 隐空间做）都是为此。

Classifier-Free Guidance（CFG）：训练时随机丢弃条件（10% 概率），采样时

$$\tilde{\epsilon} = \epsilon_\theta(x_t, \varnothing) + s\cdot(\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing))$$

$s > 1$ 增强条件一致性但降低多样性。这是 A3 DDPM 笔记本的最后一节。

22.6 四种生成模型的对照

	似然	采样速度	样本质量	训练稳定性	代表
自回归	精确	极慢（逐像素）	高	高	PixelCNN、ImageGPT
VAE	下界	快（一次前向）	偏模糊	高	VQ-VAE、SD 的 VAE
GAN	无	快	锐利	**低**	StyleGAN、CycleGAN
扩散	下界（可算）	慢（多步）	**最高**	中	DDPM、DiT、SD

22.7 常见误区

误区	纠正
"VAE 的 ELBO 是似然"	是下界；差距就是 KL 项
"GAN 的判别器越强越好"	太强会让生成器梯度消失，需要平衡
"扩散模型在学数据分布的形状"	它学的是**score/噪声**，等价于学 $\nabla_x \log p(x)$（推断：score-based 视角）
"CFG 的 s 越大越好"	过大会过饱和、失去多样性

第 23 章 3D 视觉概览

23.1 五种形状表示

表示	形式	优点	缺点
深度图（depth map）	$H \times W$	易获取（RGB-D）、可微	只有 2.5D，单视角
体素（voxel）	$D \times H \times W$ 占据栅格	卷积可直接用	分辨率立方级爆炸
点云（point cloud）	$N \times 3$	原始、无量化误差	无序、稀疏
网格（mesh）	顶点 + 面	工业标准、轻量	拓扑不规则，难学习
隐式表面（implicit）	$f(x,y,z) \to$ 占据/密度	连续、无限分辨率	需采样，训练慢

23.2 从多视角到 3D

MVCNN：12 个视角各过 CNN，在特征层做 **view pooling**（逐元素 max），再分类。讲义的实验表显示：12 视角 MVCNN 在 ModelNet40 上 89.9%，比单视角方案高 10+ 个点。

NeRF（课程在 "Neural implicit representations" 里提到）：用一个 MLP 表示 $f(x,y,z,\theta,\phi) \to (\mathrm{RGB}, \sigma)$，沿光线积分渲染像素，用多视角照片监督。核心公式：

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\,\sigma(\mathbf{r}(t))\,\mathbf{c}(\mathbf{r}(t),\mathbf{d})\,dt, \qquad T(t) = \exp\left(-\int_{t_n}^{t} \sigma\,ds\right)$$

单目深度估计的尺度歧义：小近物体与大远物体在图像上完全一样，所以绝对深度不可知，只能用**尺度不变损失**（scale-invariant loss）监督。

23.3 常见误区

误区	纠正
"体素是 3D 的终极表示"	分辨率 128^3 就是 200 万格，太贵
"NeRF 只能做静态场景"	后续工作扩展到动态/生成（推断）
"3D 任务都需要 3D 数据"	单图/多视图重建可用 2D 监督

第 24 章 视觉–语言与基础模型

24.1 从"一个任务一个模型"到"一个模型多个任务"

讲义开篇对比了两种范式：

```
传统：Data1+Task1 -> Model1 ... Data4+Task4 -> Model4
基础模型：大规模预训练 -> 一个模型 -> zero-shot / few-shot / finetune 到多任务
```

基础模型的标志：**对多任务通用/鲁棒**；常见特征：大数据 + 自监督预训练目标。

24.2 CLIP：对比学习把图像和文本对齐

数据：~400M 图文对（讲义明确给了这个量级，并引 "Demystifying CLIP Data" 说明可复现）。

训练：一个 batch 里 N 个图文对，图像编码器与文本编码器各出 N 个向量，做 $N \times N$ 的相似度矩阵，**对角线是正样本**，其余 $N^2 - N$ 个是负样本，两个方向（图→文、文→图）各算一次交叉熵。

$$\mathcal{L} = \frac{1}{2}(\mathcal{L}_{\text{img}\to\text{txt}} + \mathcal{L}_{\text{txt}\to\text{img}})$$

Zero-shot 分类：把类别名写成 prompt（"a photo of a {class}"），算文本嵌入，取相似度最高的类。**prompt engineering 在 CLIP 上收益很大**（讲义提到这是原论文的发现）。

局限：对细粒度分类、计数、空间关系弱；检索性能好但生成能力无。

24.3 视觉–语言模型（VLM）的三种接法

类型	做法	代表
对比式（双塔）	各自编码，对齐嵌入空间	CLIP、ALIGN

类型	做法	代表
融合式	交叉注意力深度融合	Flamingo、CoCa
拼接式（LM + 视觉编码器）	视觉 token 当前缀喂给 LLM	LLaVA、Qwen-VL

讲义第 16 讲把这三类与 "Chaining（LM + CLIP + 工具）"、"其他（SAM、Whisper、DALL·E）"并列，说明课程把它当作"基础模型的分类学"来讲，而不是逐个推导。

24.4 SAM：可提示分割

用"prompt（点/框/掩码）+ 图像"输出掩码，在 11M 图像、1.1B 掩码上训练。它的意义是把分割变成**可提示的通用任务**（推断：课程只列为"基础模型"一例，未展开）。

24.5 常见误区

误区	纠正
"CLIP 能生成图像"	它只做对齐/检索/分类，生成要靠扩散模型
"zero-shot 一定弱于有监督"	在分布外/新类别上 zero-shot 常更强
"VLM = CLIP + LLM"	关键在**视觉 token 如何接入**与**指令数据**的质量

第 25 章 与已有笔记的衔接

25.1 与 CS229：把"统计学习"接到"视觉工程"

CS229 的概念	在 CS231n 里的形态	差异点
偏差-方差权衡	小模型欠拟合 / 大模型 + 强正则过拟合	视觉里的"正则"多了 dropout、BN、增强、TTA
L1/L2 正则	weight decay、稀疏特征选择	视觉里 L2 更常用；L1 多用于剪枝/稀疏
逻辑回归 / softmax	**完全相同的损失**，只是输入从特征变成像素	多了"模板可视化"这一层解释
核方法 / SVM 对偶	多类 hinge loss（原问题形式）	课程只讲 primal，不讲对偶
生成式 vs 判别式	VAE/GAN/扩散 vs 分类器	生成模型的工程细节远超 CS229 的 GMM/EM
EM	VAE 的变分推断是它的"神经网络版"	用重参数化 + SGD 代替 E 步闭式解

最重要的衔接：CS229 教你"给定特征 $\boldsymbol{x}$，如何学 $\boldsymbol{y}$"；CS231n 教你"**如何连特征一起学**"。所以 CS229 的模型评估、正则化、偏差方差分析全部适用，只是"特征工程"这一环被卷积网络替掉了。

25.2 与 CS336：Transformer 从语言搬到图像

CS336 的内容	在 CS231n 的对应	需要注意的差异
attention 实现、KV cache	第 16 章、A3 的 `MultiHeadAttention`	视觉里序列长度 $N=(H/P)^2$，通常远小于语言
RMSNorm / pre-LN	第 16 章明确讲 pre-LN 与 RMSNorm	视觉 ViT 也用同一套
训练系统、显存核算	第 20 章分布式训练	视觉多了"大 batch + 强增强"这一维度
scaling law	未讲	视觉的 scaling 更多体现在数据（JFT-300M）而非纯参数
tokenizer	无	视觉用 patch / VQ-VAE codebook 代替

一个值得注意的对称性：语言模型的"预测下一个 token"和 MAE 的"重建被遮 patch"是同一类自监督思想，只是冗余度不同导致掩码率差 5 倍（15% vs 75%）。

25.3 与 CS285（RL）：视觉作为状态

RL 里的需求	视觉提供的能力	风险
从像素学策略	CNN/ViT 编码器 + 特征	样本效率极低
世界模型	视频预测、扩散生成	长时预测误差累积
sim-to-real	域随机化 + 数据增强	视觉外观差异是主要 gap
表征学习	自监督预训练（DINO/MAE）	表征好 ≠ 控制好

（推断） 视觉在 RL 里的核心矛盾是：RL 需要**因果/可控**的表征，而自监督视觉学到的多是**外观**表征。这也是讲义把 "World Modeling" 与 "Embodied Intelligence" 放在课程末尾（而不是核心）的原因。

25.4 一条建议的学习顺序

```
CS229 -> CS231n 第 1-9 章（分类器 + 反传）-> 第 10-14 章（CNN + 训练）
      -> CS336（Transformer 深入）-> CS231n 第 16-24 章（注意力/检测/生成/3D/VLM）-> CS285
```

第 26 章 复习清单

26.1 必须能默写（12 项）

- [ ] 卷积输出尺寸 $H_{out} = \lfloor (H + 2P - K)/S \rfloor + 1$；感受野递推 $r_{out} = r_{in} + (K-1)j_{in}$，$j_{out} = j_{in} S$
- [ ] 多类 hinge loss 及其梯度（"计数"技巧）；softmax + CE 的 loss 与梯度 $p - y$
- [ ] BN 的 forward（train/test 两套）与反向闭式；Xavier / He 初始化的方差推导
- [ ] SGD / Momentum / RMSProp / Adam 的更新式（含偏差校正）
- [ ] 反向传播"上游 × 局部"法则与四种门；Affine / ReLU / Conv / MaxPool 的 fwd–bwd 对
- [ ] ResNet 的 $H(x) = F(x) + x$ 与梯度 $1 + \partial F/\partial x$；自注意力与 $\sqrt{d_k}$；IoU / AP / mAP

26.2 必须能解释"为什么"（10 项）

- [ ] kNN 为什么在像素上失效？（三种变换同距离）；Δ 为什么与 λ 冗余
- [ ] 为什么用三个 3×3 而不是一个 7×7；ResNet 的退化为什么不是过拟合
- [ ] BN 测试时为什么不能用 batch 统计；dropout 为什么要除以 $1 - p$
- [ ] Adam 为什么需要偏差校正；attention 为什么要除以 $\sqrt{d_k}$
- [ ] 扩散为什么预测噪声而不直接预测 x_0；mAP 为什么跨数据集不可比

26.3 必须见过的实验现象（8 项）

- [ ] 初始化 std 太小 → 激活逐层趋 0；太大 → 爆炸；深层 plain 网络的**训练误差**反而更高
- [ ] 随机搜索 W 在 CIFAR-10 上约 15.5%；初始 softmax loss $\approx \log C$；kNN（像素）~28%、特征+kNN ~40%、CNN ~95%
- [ ] 过拟合小样本时 loss 应到 0；对抗样本肉眼无变化但分类错；CAM 热力图与物体位置对应

第 27 章 自测题（附答案）

Q1. $32 \times 32 \times 3$ 输入，64 个 3×3 滤波器，stride 2，pad 1，输出尺寸与参数量？

> A. $H_{out} = \lfloor (32 + 2 - 3)/2 \rfloor + 1 = 16$，输出 $64 \times 16 \times 16$；参数 $64 \times (3 \times 3 \times 3 + 1) = 1792$。

Q2. 三分类，某样本分数 $s = [1, 3, 2]$，真值 $y = 2$（索引从 0 起，$s_1 = 3$）。hinge loss（$\Delta = 1$）与 softmax loss 各是多少?

> **A.** hinge：$\max(0, 1-3+1) + \max(0, 2-3+1) = 0 + 0 = 0$。softmax：$p = [e^1, e^3, e^2]/Z$，$Z = 2.718 + 20.09 + 7.389 = 30.19$，$p_1 = 0.665$，$L = -\log 0.665 = 0.407$。**注意：hinge 已经为 0 时 softmax 仍不为 0**，因为 softmax 永远想更自信。

Q3. 写出 softmax 对分数的梯度，并说明为什么它"自带数值稳定问题"。

> **A.** $\partial L/\partial s_j = p_j - \mathbb{1}[j = y]$。$p$ 由 `exp` 得到，分数很大时会溢出；解法是减行最大值（不改变结果，只改变数值范围）。

Q4. BN 在训练和测试时的行为差异？如果测试时错误地用了 batch 统计，会发生什么?

> **A.** 训练用当前 batch 的 μ_B, σ_B 并更新 running stats；测试用 running stats。若测试用 batch 统计：batch=1 时 $\hat{x} = 0$，输出退化为 β，预测全错；batch 较大时输出随 batch 组成变化，不确定。

Q5. 一个 5 层网络，每层 3×3 conv（stride 1, pad 1），感受野多大？若中间插了 2 个 2×2/stride 2 的池化呢?

> **A.** 无池化：$1 + 5 \times 2 = 11$。插入 2 次 stride 2 后需逐层递推：设 conv 与 pool 交替，j 会翻倍两次，最终感受野约为 $1 + 2 + 2 + (2 \times 2) + (2 \times 2) + (2 \times 4) + \ldots$，**应逐层用递推公式算，不能心算**——这正是考试要考"会不会用递推"。

Q6. 为什么 ResNet 的残差连接能缓解梯度消失？写出梯度表达式。

> **A.** $H(x) = F(x) + x \Rightarrow \partial \mathcal{L}/\partial x = (\partial \mathcal{L}/\partial H)(1 + \partial F/\partial x)$。恒等项的 $+1$ 保证梯度至少原样回传，不会因 $\partial F/\partial x$ 连乘而衰减到 0。

Q7. Adam 的偏差校正公式，以及漏掉它会怎样?

> **A.** $\hat{m} = m/(1-\beta_1^t)$，$\hat{v} = v/(1-\beta_2^t)$。漏掉时初期 m, v 都被 0 拉小，但 $\sqrt{v}$ 相对更小，导致**有效步长偏大且不稳定**（尤其前 10 步）；实践中表现为 loss 初期震荡。

Q8. 检测里 AP 的计算步骤？为什么 COCO 的 AP 与 VOC 的 AP 不能直接比?

> **A.** 按置信度排序 → 累加 TP/FP → 算 precision/recall 曲线 → 曲线下面积。COCO 在 IoU 0.5:0.95 上平均且区分物体大小，VOC 只用 0.5（或 0.75），阈值与匹配规则不同，数值不可比。

Q9. SimCLR 的损失里，温度 τ 的作用？τ 很大会怎样?

> **A.** τ 控制 softmax 的锐度。τ 大 → 分布平坦 → 所有负样本权重接近 → 学不到判别性；τ 小 → 只关注最难的负样本 → 训练不稳定。

Q10. DDPM 的训练目标为什么可以简化为"预测噪声"的 MSE?

> **A.** 在 $q(x_t|x_0)$ 的闭式下，$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$，预测 x_0 与预测 ϵ 一一对应（线性变换）；代入 ELBO 后，忽略与 t 相关的权重系数就得到 L_{simple}。这个简化牺牲了理论最优性，但换来更好的样本质量（推断：这是 DDPM 论文的实证结论）。

Q11. 为什么 ViT 需要位置编码，而 CNN 不需要?

> **A.** 自注意力对输入顺序是**置换等变**的（打乱 patch 顺序，输出只是同样打乱），不加位置编码就无法区分"猫在左上"和"猫在右下"。CNN 的卷积核在空间上滑动，位置信息由"哪个位置的输出"隐式携带。

Q12. 一个训练任务：train acc 99%，val acc 70%。列出至少四种干预手段，并说明优先级。

> **A.** ① 数据增强（最高优先，成本最低）；② weight decay；③ dropout；④ 早停；⑤ 收集更多数据；⑥ 减小模型（最后考虑）。**不要**先加层或加训练轮数。

第 28 章 参考来源

> 所有链接均在 **2026-11-02** 抓取并验证可访问。标注 🔒 的需要 Stanford 账号。

28.1 课程官方页面

资源	URL	状态
课程主页（Spring 2026）	https://cs231n.stanford.edu/	✅
当季日程（含每讲 PDF 链接）	https://cs231n.stanford.edu/schedule.html	✅
作业页（A1 12% / A2 18% / A3 15%）	https://cs231n.stanford.edu/assignments.html	✅
2025 版主页	https://cs231n.stanford.edu/2025/	✅
2025 版日程	https://cs231n.stanford.edu/2025/schedule.html	✅
2017 版 syllabus（含 YouTube 播放列表）	https://cs231n.stanford.edu/2017/syllabus.html	✅
课程视频（当季）	Canvas → Panopto Course Videos	🔒

28.2 讲义 PDF（逐讲下载并 `pdftotext -layout` 阅读）

统一路径为 `https://cs231n.stanford.edu/slides/2026/lecture_{N}.pdf`，`N=2..16` 共 15 个文件全部返回 200 并成功转文本；L1 例外，只有 `lecture_1_part_1.pdf` 与 `lecture_1_part_2.pdf` 两个文件（`lecture_1.pdf` 为 404）。2025 版同构路径为 `/slides/2025/lecture_{N}.pdf`，其中 L1 同样拆成两部分。

讲	本地文本行数（`pdftotext -layout`）	讲	本地文本行数
L1（两部分）	1127 + 622	L9	2447
L2	1045	L10	1428
L3	1574	L11	3033
L4	1817	L12	1435
L5	1456	L13	1911
L6	1881	L14	2372
L7	1959	L15	1108
L8	2660	L16	1674

> 说明：L1 在 schedule 页拆成 `part_1` / `part_2`；L17（World Modeling）与 L18（Human-Centered AI）在当季 schedule 上**没有挂公开 PDF 链接**，故本笔记未逐页阅读这两讲，相关段落只写了日程页给出的标题。

28.3 官方长文（Useful Notes）

笔记	URL	用在哪
图像分类	https://cs231n.github.io/classification/	第 3 章
线性分类	https://cs231n.github.io/linear-classify/	第 4–5 章
优化 I（梯度）	https://cs231n.github.io/optimization-1/	第 7 章
优化 II（反向传播）	https://cs231n.github.io/optimization-2/	第 9 章
神经网络 I/II/III	https://cs231n.github.io/neural-networks-1/ （及 `-2`、`-3`）	第 8、12、13 章
卷积网络	https://cs231n.github.io/convolutional-networks/	第 10–11 章
理解 CNN	https://cs231n.github.io/understanding-cnn/	第 18 章

笔记	URL	用在哪
迁移学习	https://cs231n.github.io/transfer-learning/	第 14 章
注意力 / Transformer	https://cs231n.github.io/attention/ ；https://cs231n.github.io/transformers/	第 16 章
生成模型	https://cs231n.github.io/generative-models/	第 22 章

28.4 作业与代码

资源	URL	状态
cs231n 组织	https://github.com/cs231n	✅
课程笔记/作业仓库	https://github.com/cs231n/cs231n.github.io	✅
2026 作业 1 说明	https://github.com/cs231n/cs231n.github.io/blob/master/assignments/2026/assignment1.md	✅
2026 作业 2 说明	同上 `assignment2.md`	✅
2026 作业 3 说明	同上 `assignment3.md`	✅
起始代码	同上 `assignment{1,2,3}.zip`（含 `.ipynb` 与 `cs231n/` 包）	✅

28.5 讲义中引用的原始论文（仅列课程明确提及者）

主题	论文（标题与年份取自讲义页引用文字）
卷积网络起源	LeCun et al., *Gradient-based learning applied to document recognition*, 1998；Fukushima, *Neocognitron*, 1980
深度分类	Krizhevsky et al., *ImageNet Classification with Deep CNNs*, NIPS 2012；Simonyan & Zisserman, *Very Deep Convolutional Networks*, 2014；Szegedy et al., *Going Deeper with Convolutions*, 2014
残差与初始化	He et al., *Deep Residual Learning*, 2015；He et al., *Delving Deep into Rectifiers*, ICCV 2015；Ioffe & Szegedy, *Batch Normalization*, 2015
正则与调参	Srivastava et al., *Dropout*, JMLR 2014；Bergstra & Bengio, *Random Search for Hyper-Parameter Optimization*, 2012；Goyal et al., *Accurate, Large Minibatch SGD*, 2017

主题	论文（标题与年份取自讲义页引用文字）
序列与注意力	Vaswani et al., *Attention Is All You Need*, NIPS 2017；Bahdanau et al., *Neural Machine Translation by Jointly Learning to Align and Translate*, ICLR 2015；Dosovitskiy et al., *An Image is Worth 16x16 Words*, ICLR 2021
分割	Long et al., *Fully Convolutional Networks*, CVPR 2015；Ronneberger et al., *U-Net*, 2015；He et al., *Mask R-CNN*, ICCV 2017
检测	Girshick et al., *Rich feature hierarchies*（R-CNN）, CVPR 2014；Ren et al., *Faster R-CNN*, 2015；Redmon et al., *You Only Look Once*, CVPR 2016；Liu et al., *SSD*, ECCV 2016
可视化	Zhou et al., *Learning Deep Features for Discriminative Localization*（CAM）, CVPR 2016；Simonyan et al., *Deep Inside Convolutional Networks*, 2014
视频	Karpathy et al., *Large-scale Video Classification*, CVPR 2014
自监督	Chen et al., *SimCLR*, ICML 2020；He et al., *MoCo*, CVPR 2020；Caron et al., *DINO*, 2021；Oquab et al., *DINOv2*, 2023
生成	Kingma & Welling, *Auto-Encoding Variational Bayes*, 2014；Goodfellow et al., *Generative Adversarial Nets*, 2014；Ho et al., *Denoising Diffusion Probabilistic Models*, 2020
3D 与多模态	Mildenhall et al., *NeRF*, 2020；Radford et al., *CLIP*, 2021

> **引用纪律**：以上标题与年份取自讲义页的引用文字；我**没有**逐一打开原文核对页码/卷期，因此不给 DOI 或具体页码。正式引用请以论文原文为准。

28.6 "推断"标注的汇总

为避免误导，把主要推断集中列出（这些**不是**课程原文）：① VGG/AlexNet 参数量（60M / 138M）与 GoogLeNet 6.7%、AlexNet 15.3% 的赛果——公开常识，讲义页只给了 VGG 7.3%、ZFNet 11.7%、ResNet 3.57%；② DenseNet/SENet 的精度"量级"；③ Adam 默认超参；④ 深度可分离卷积、Swin、WGAN-GP、score-based 视角等课程未展开内容；⑤ 工业界工程经验（冻结 BN 统计、分层学习率、SyncBN）。

全文完。 若只记一句话：CS231n 的主线是**用可微的、带视觉先验的函数逼近"像素 → 语义"的映射**，而所有架构创新（卷积、残差、注意力、扩散）都是在回答"这个函数应该长什么样，才能既好优化又泛化"。